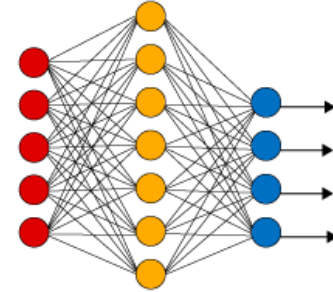
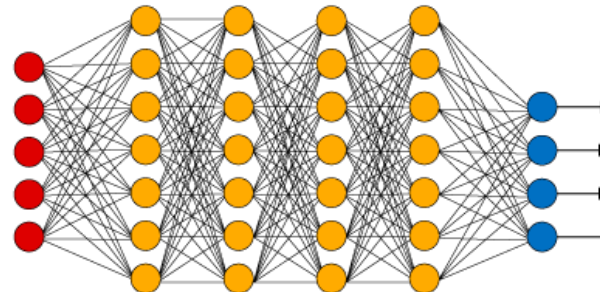


Session 1

Simple Neural Network



Deep Learning Neural Network

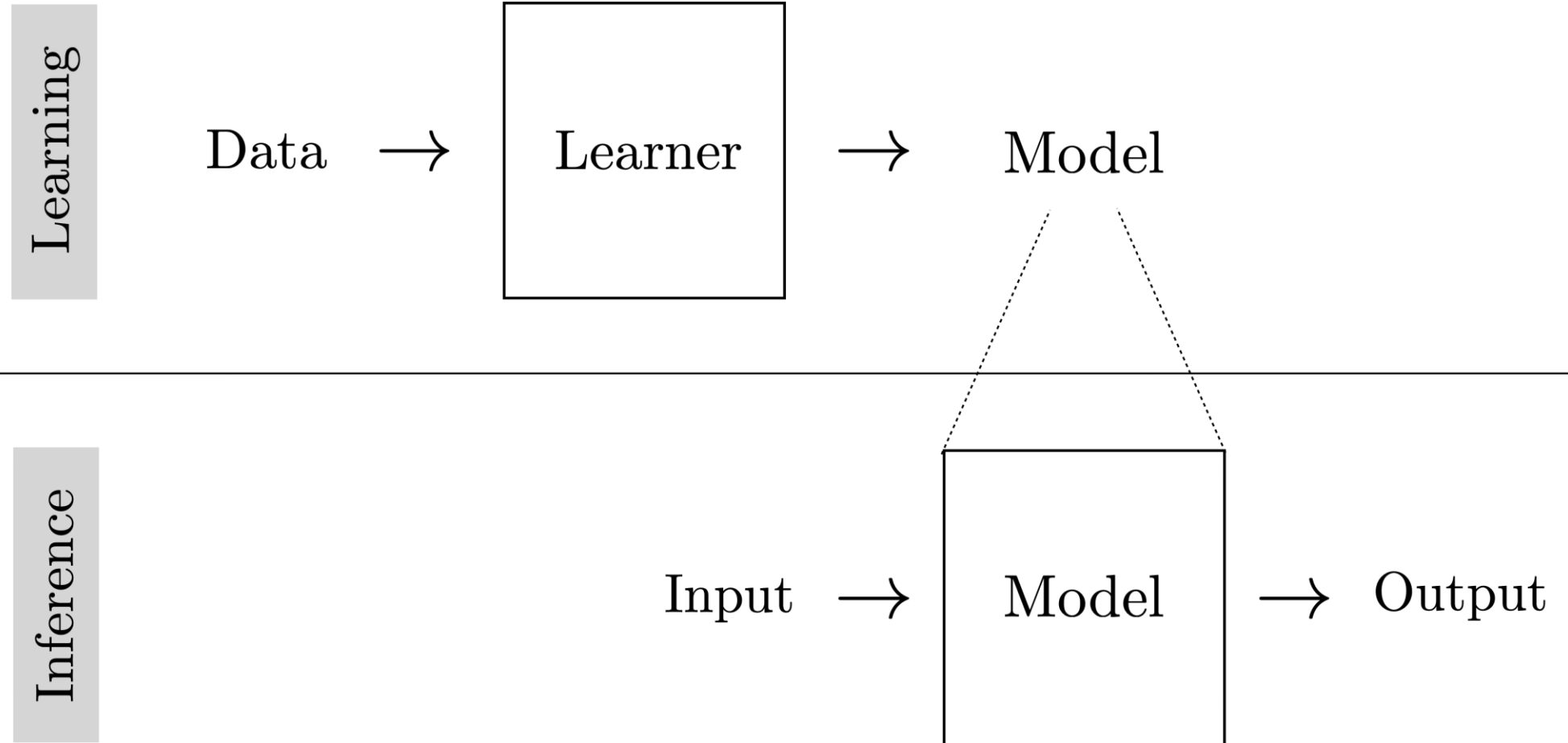


● Input Layer ● Hidden Layer ● Output Layer

Deep Learning

Introduction and Fundamentals

Learning



Learning

Learning from Examples

Learning from Rules

Learning

Learning from Examples

Learning from Rules

$$2 \star 3 = 36$$

$$7 \star 1 = 49$$

$$5 \star 2 = 100$$

$$2 \star 2 = 16$$

$$3 \star 5?$$

Learning

Learning from Examples

Learning from Rules

Training

$2 \star 3 = 36$
 $7 \star 1 = 49$
 $5 \star 2 = 100$
 $2 \star 2 = 16$



Your
brain



$x \star y \rightarrow (xy)^2$

Testing

$3 \star 5 \rightarrow$

$x \star y \rightarrow (xy)^2$

$\rightarrow 225$

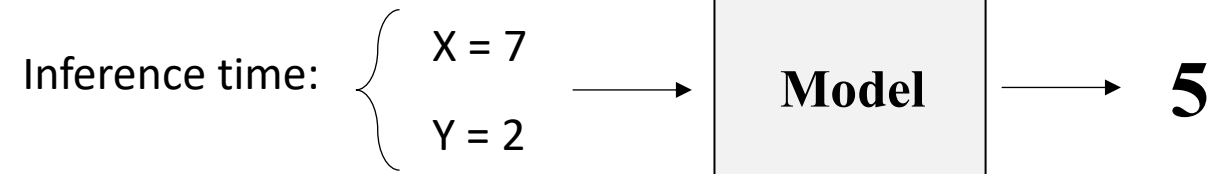
Learning

Learning from Examples

Learning from Rules

Knowledge Base:

$$\mathbf{x}, y \rightarrow y - \mathbf{x}$$



Learning

Learning from Examples

Learning from Rules

Knowledge Base:

if age > 30 then $p \times 2$

if age > 60 then $p \times 3$

Inference time:

Name: Omid

Age: 32

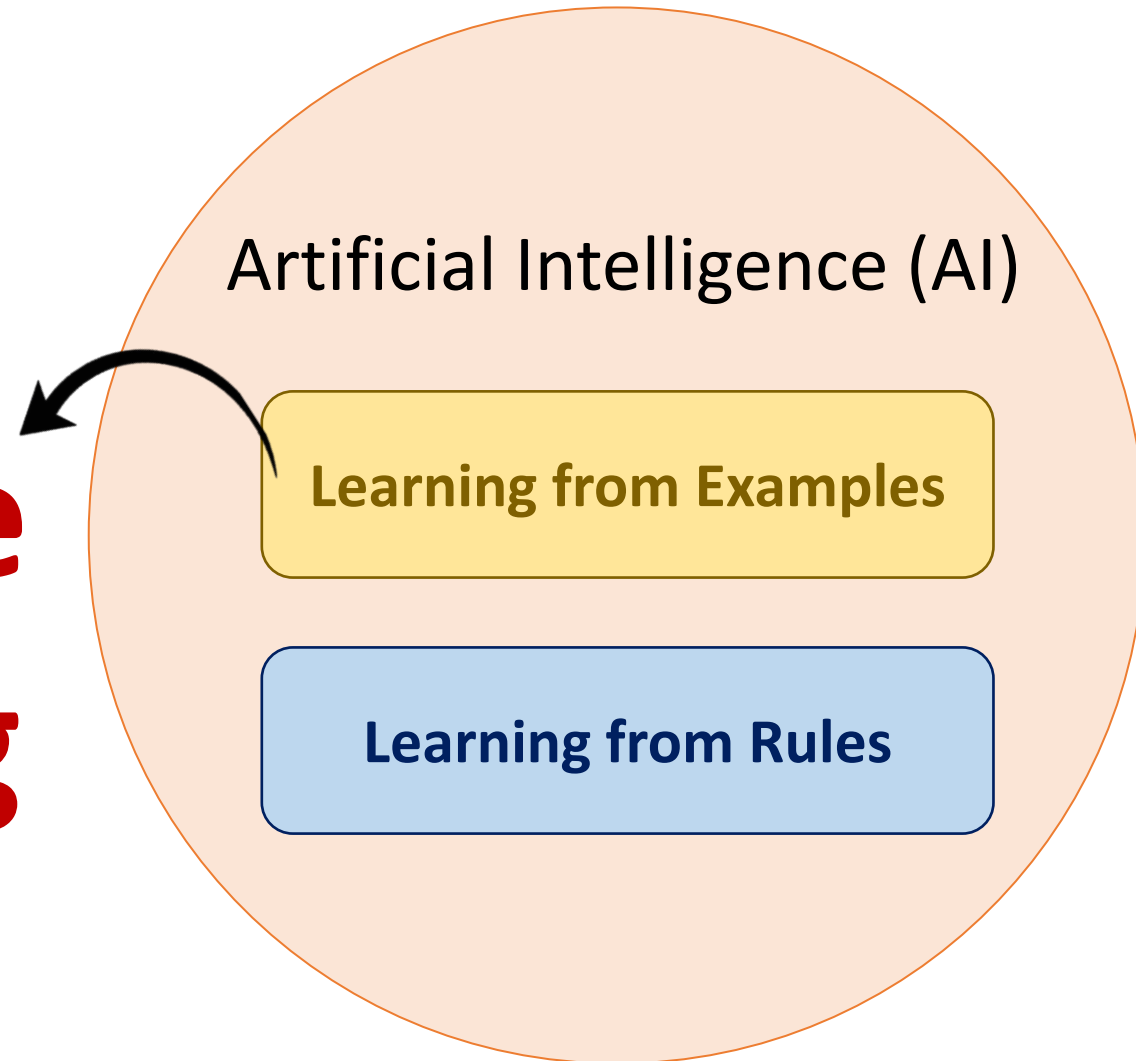
$p = 0.2$

Model

Output = 0.4

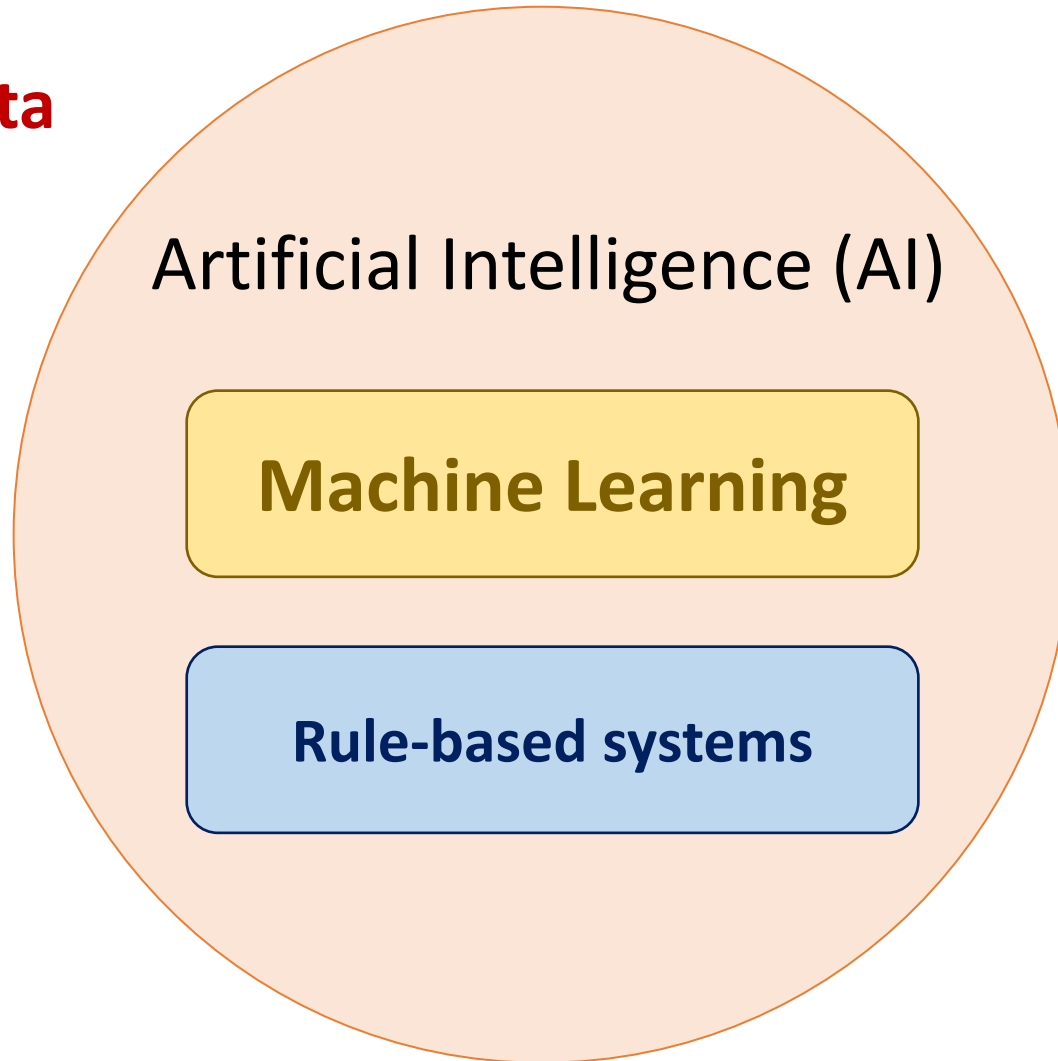
Learning

Machine Learning



Learning

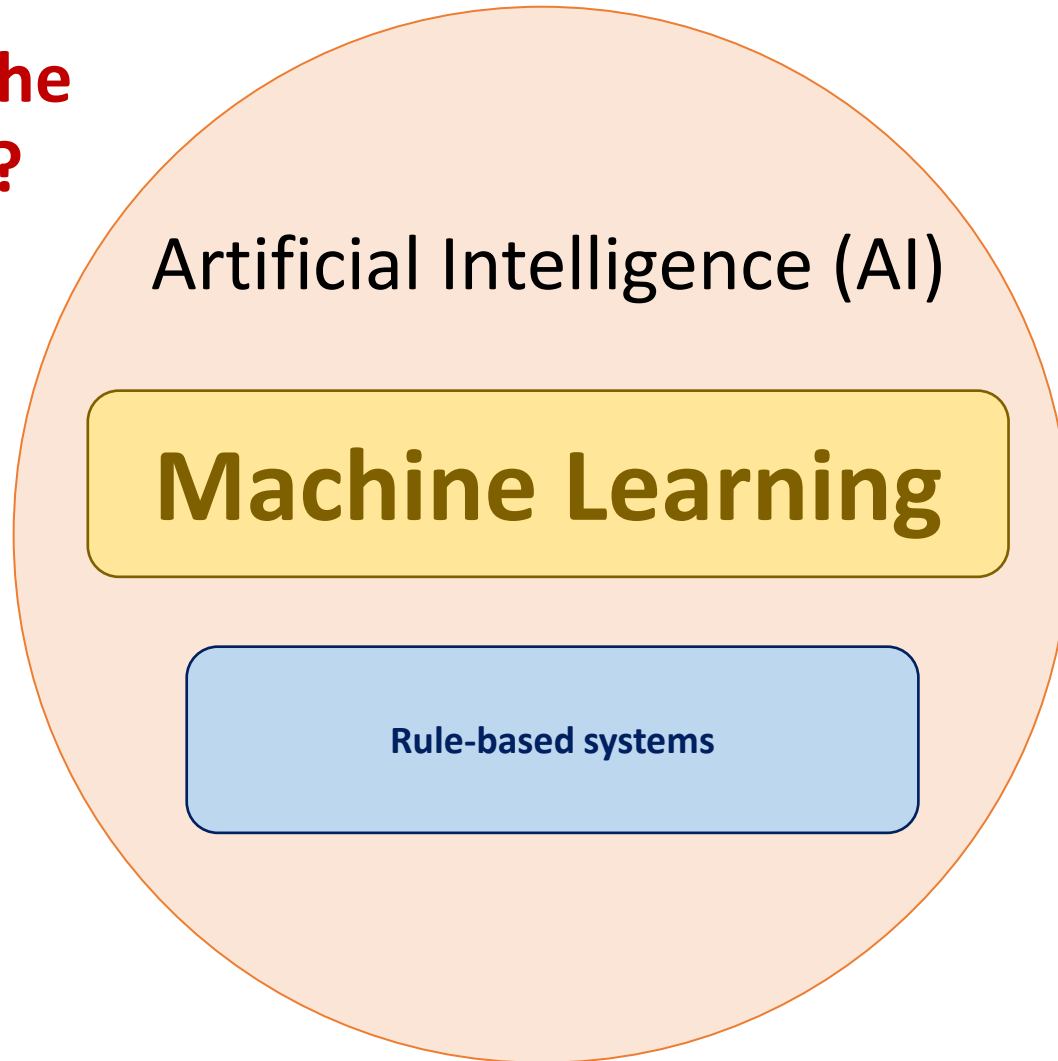
Machine Learning = Data



Learning

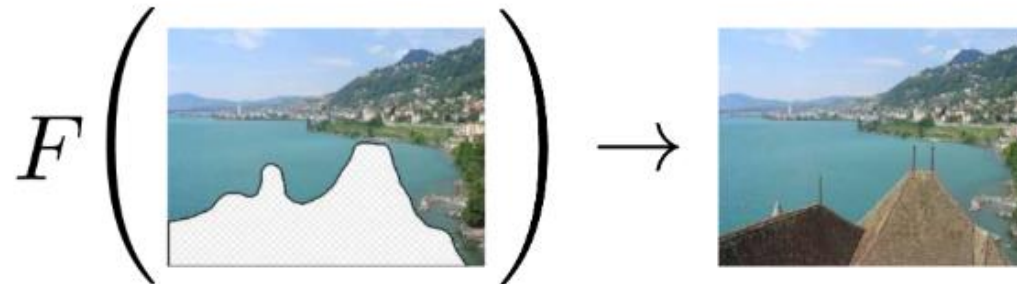
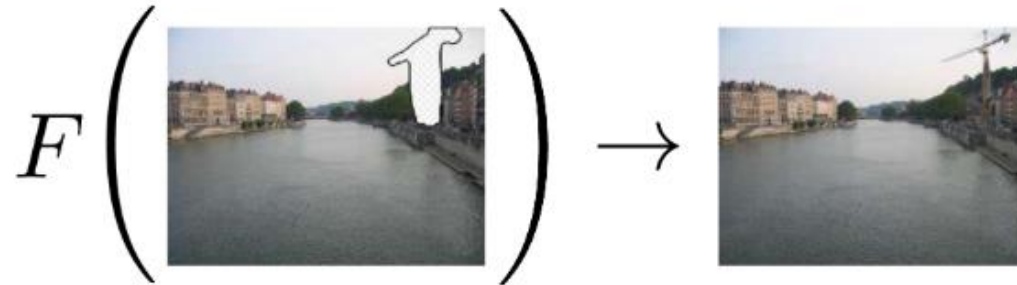
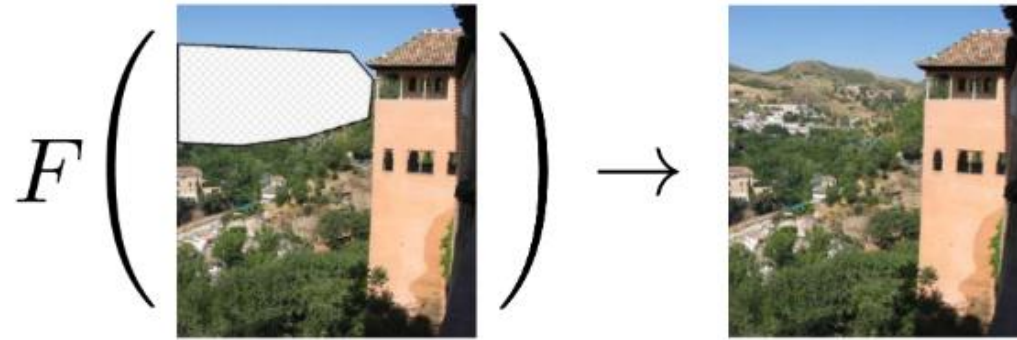
Why has ML become the dominant form of AI?

- Real-world complexity
- Generalization (Zero-shot learning)
- Data



Learning

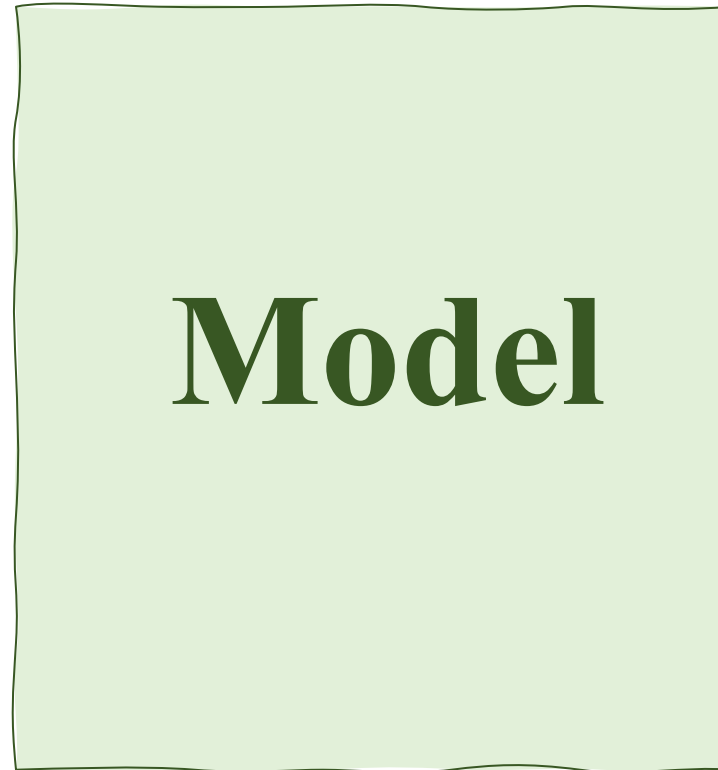
- Real-world complexity



Role of Data

Training Time

$$5 \times 2 = 100$$



Test Time

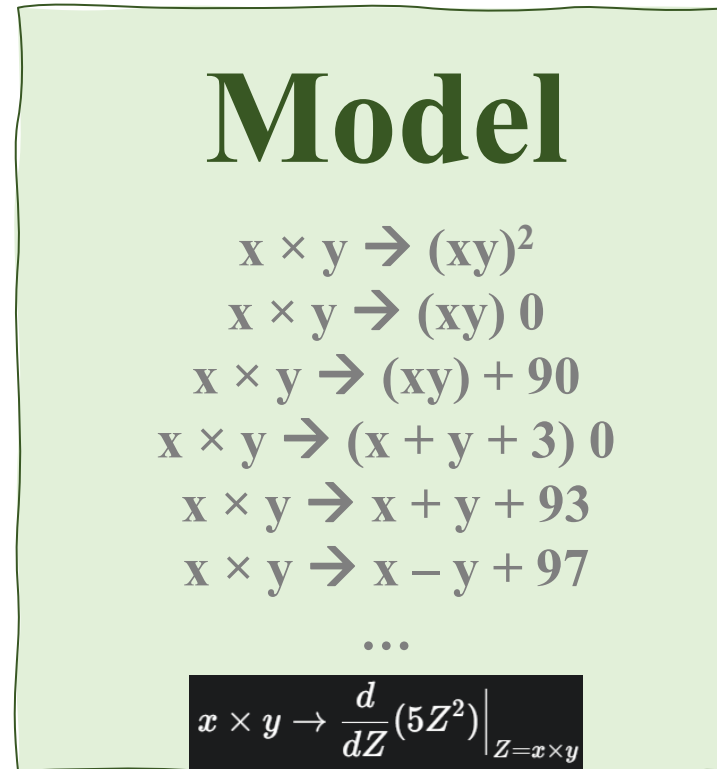


$$4 \times 6 = ?$$

Role of Data

Training Time

$$5 \times 2 = 100$$



Test Time



$$4 \times 6$$

$$\begin{aligned}&= 144 \\&= 240 \\&= 114 \\&= 130 \\&= 103 \\&= 95 \\&\dots\end{aligned}$$

Role of Data

Model

Data is important

$$5 \times 2 = 10$$

$$4 \times 6$$

= 144
= 240
= 114
= 130
= 103
= 95
...

Data Scarcity



Data



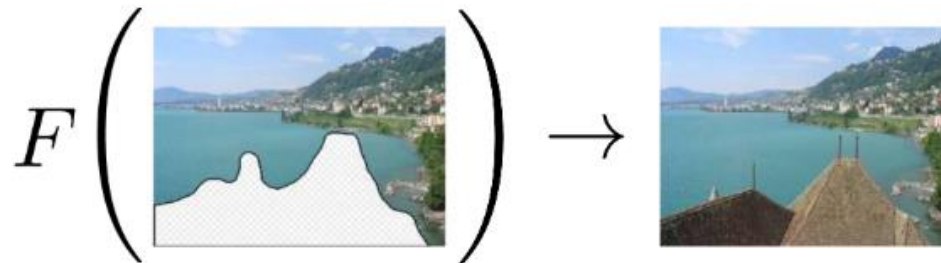
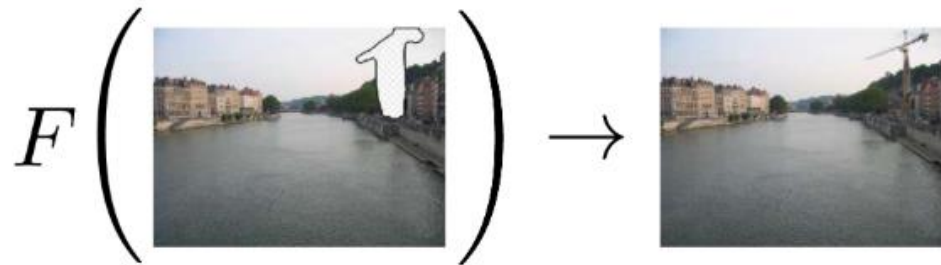
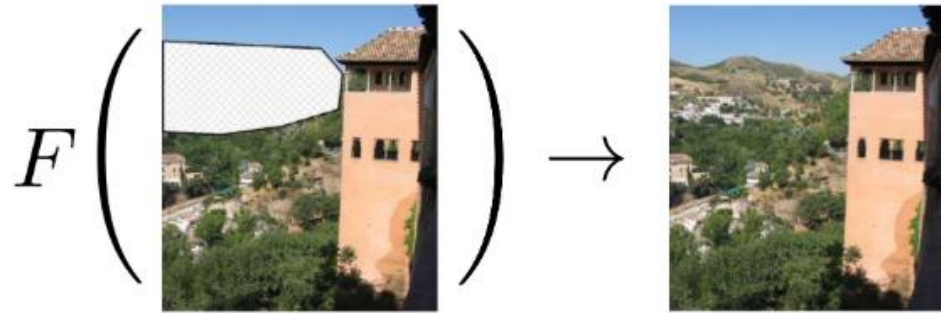


~~Data~~

(Labeled) **Data Scarcity**

Data Scarcity

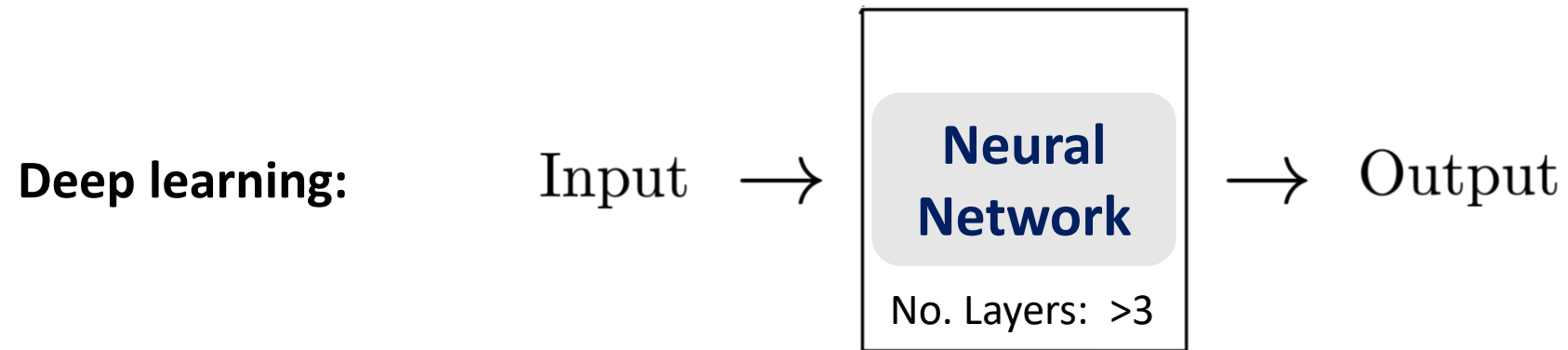
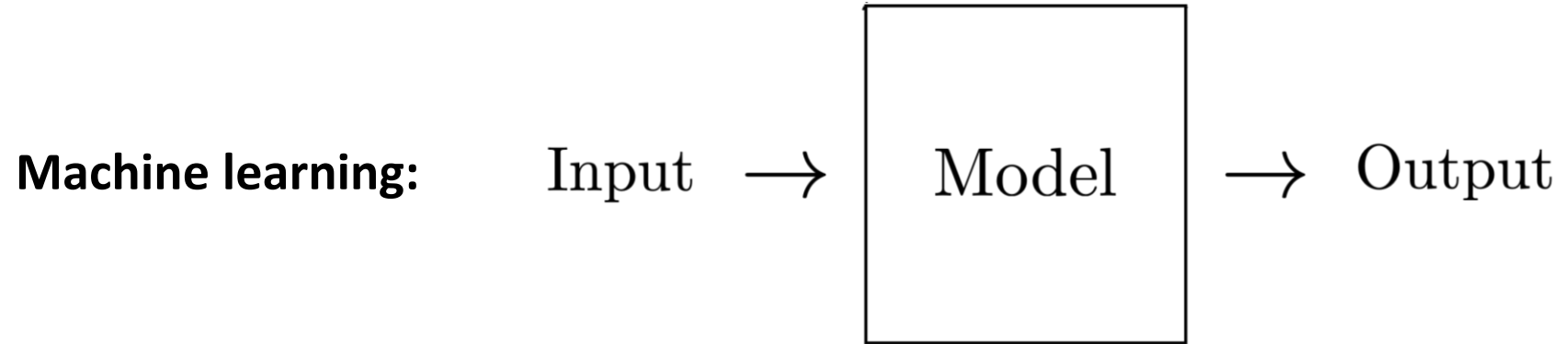
For example:



Solutions:

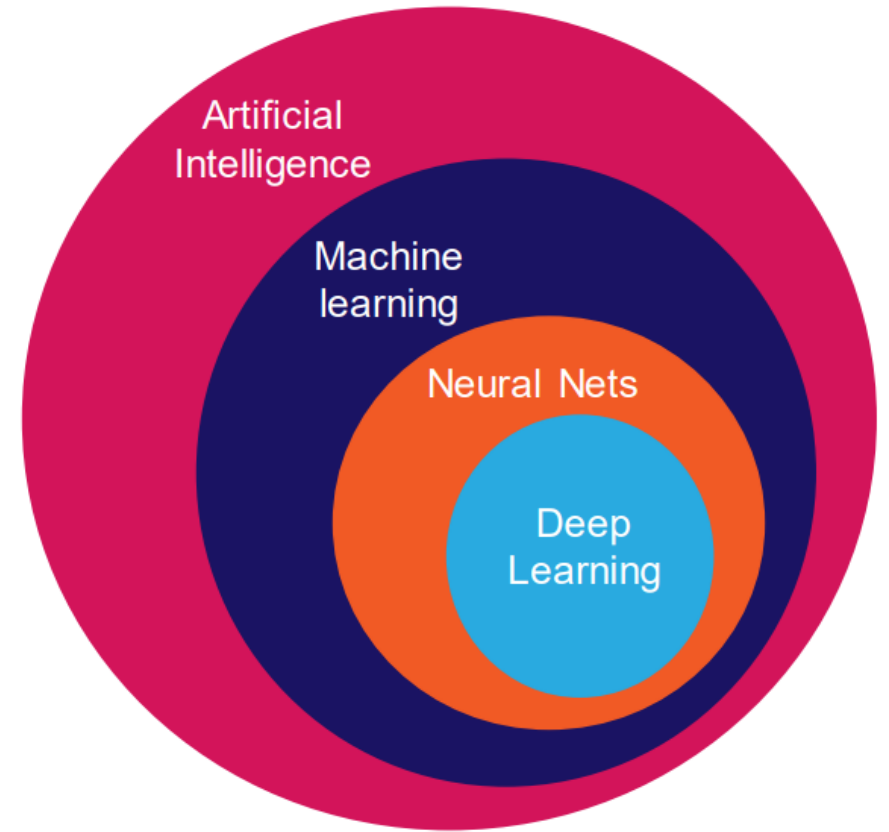
- Few-shot learning
- ? Self-supervised learning
- ? Transfer learning
- Etc.

Deep learning



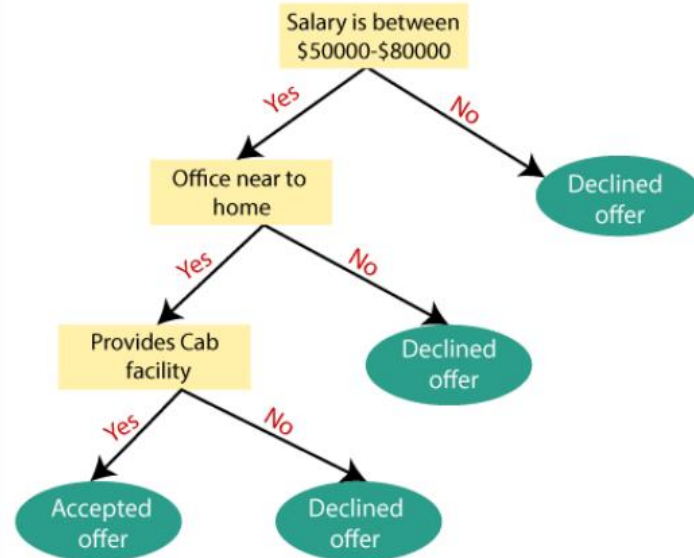
AI & ML & DL

- **Artificial intelligence** is a science like mathematics or biology. It studies ways to build intelligent programs and machines that can creatively solve problems, which has always been considered a human prerogative.
- **Machine learning** is a subset of artificial intelligence (AI) that provides systems the ability to automatically learn and iteratively improve from experience (data) without being explicitly programmed. In ML, there are different algorithms (e.g. SVM, neural networks) that help to solve problems.
- **Neural learning, or deep neural learning**, is a subset of machine learning, which uses the neural networks to analyze different factors with a structure that is similar to the human neural system.

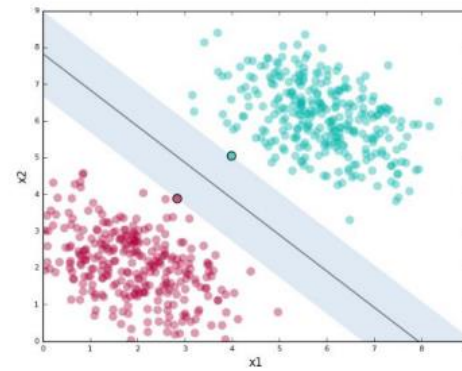


Recap

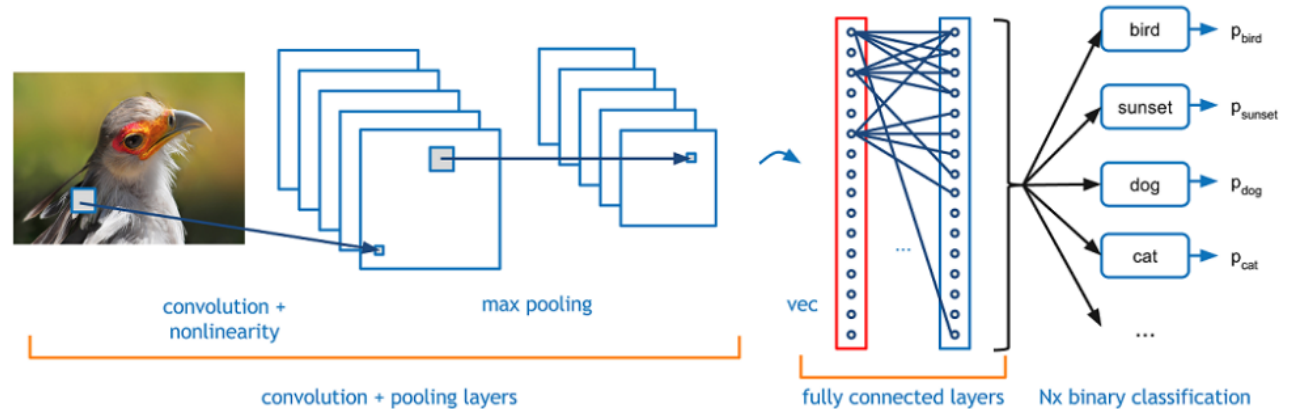
- AI: decision tree



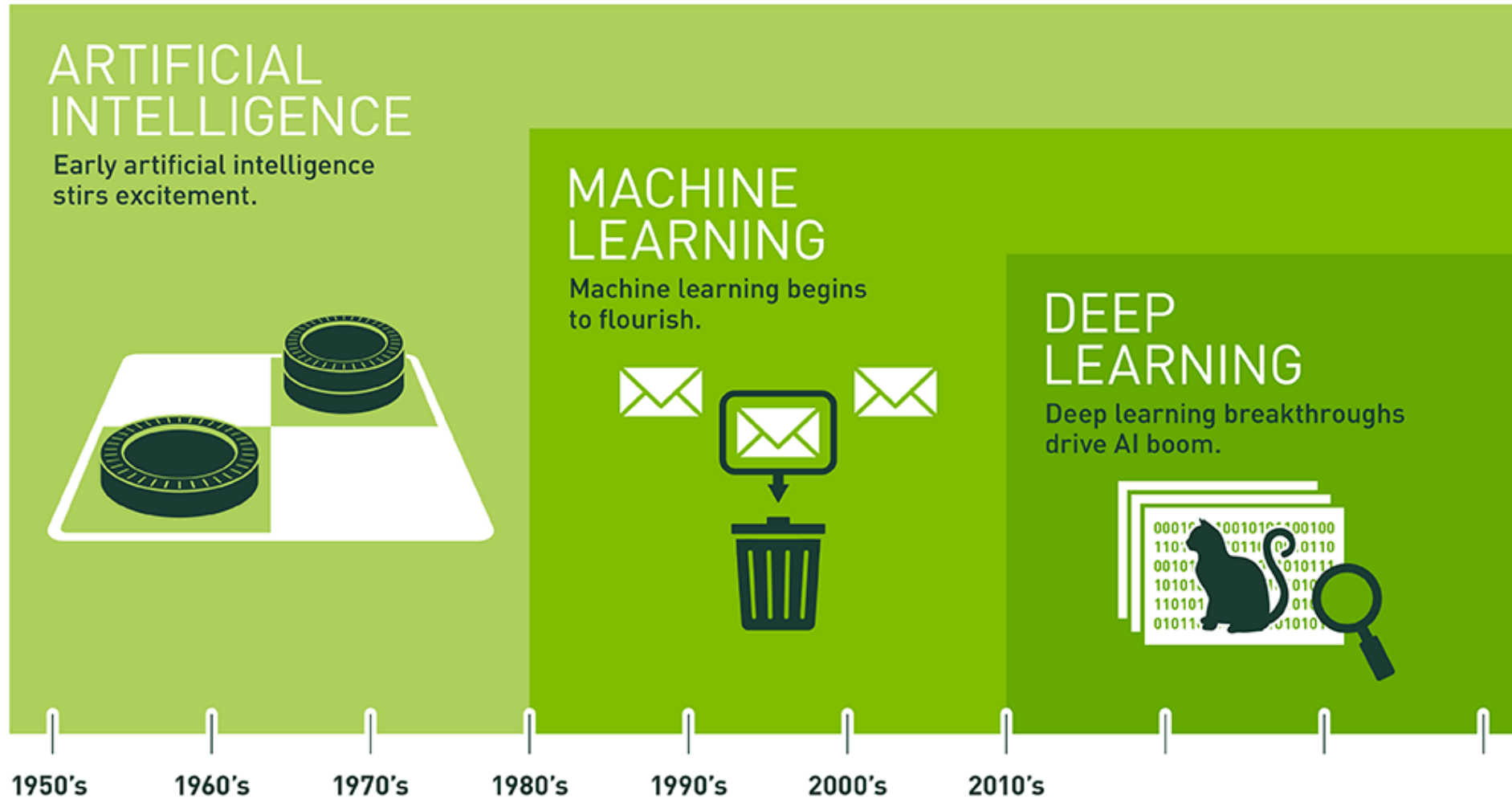
- ML: SVM



- DL: CNN

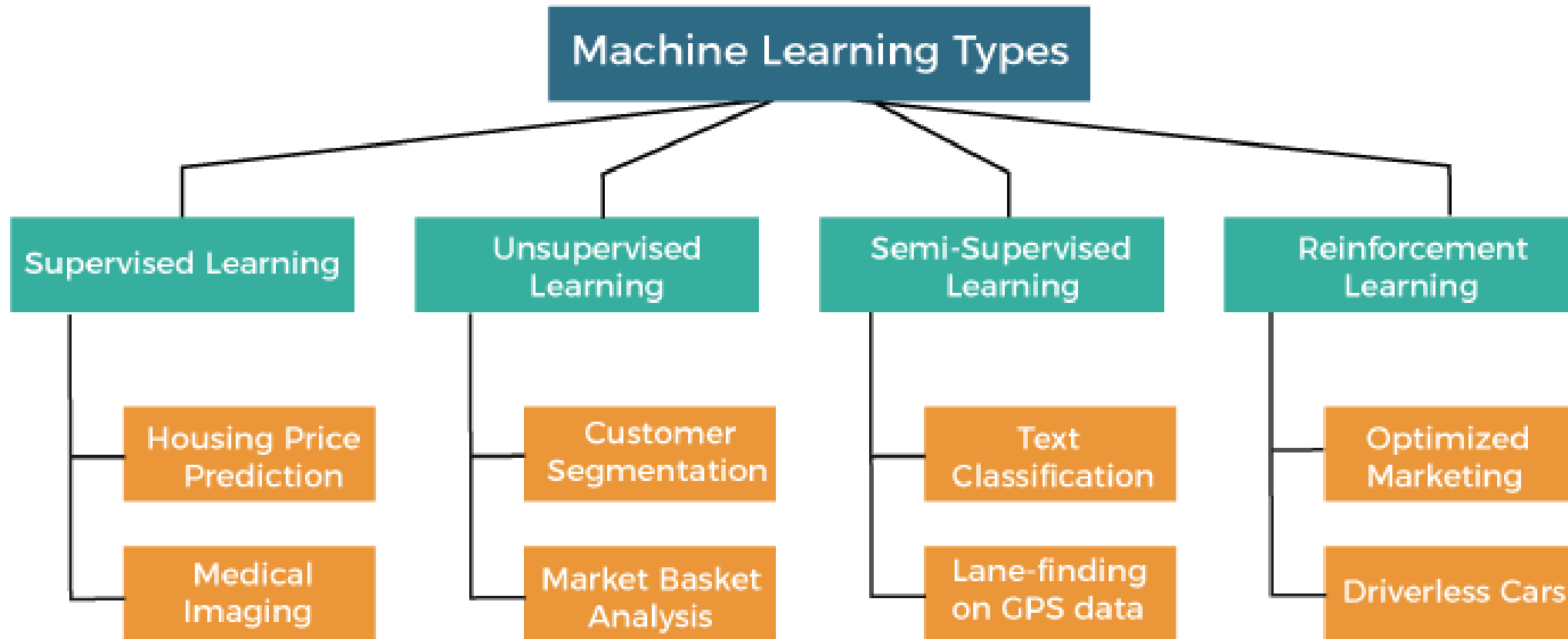


History

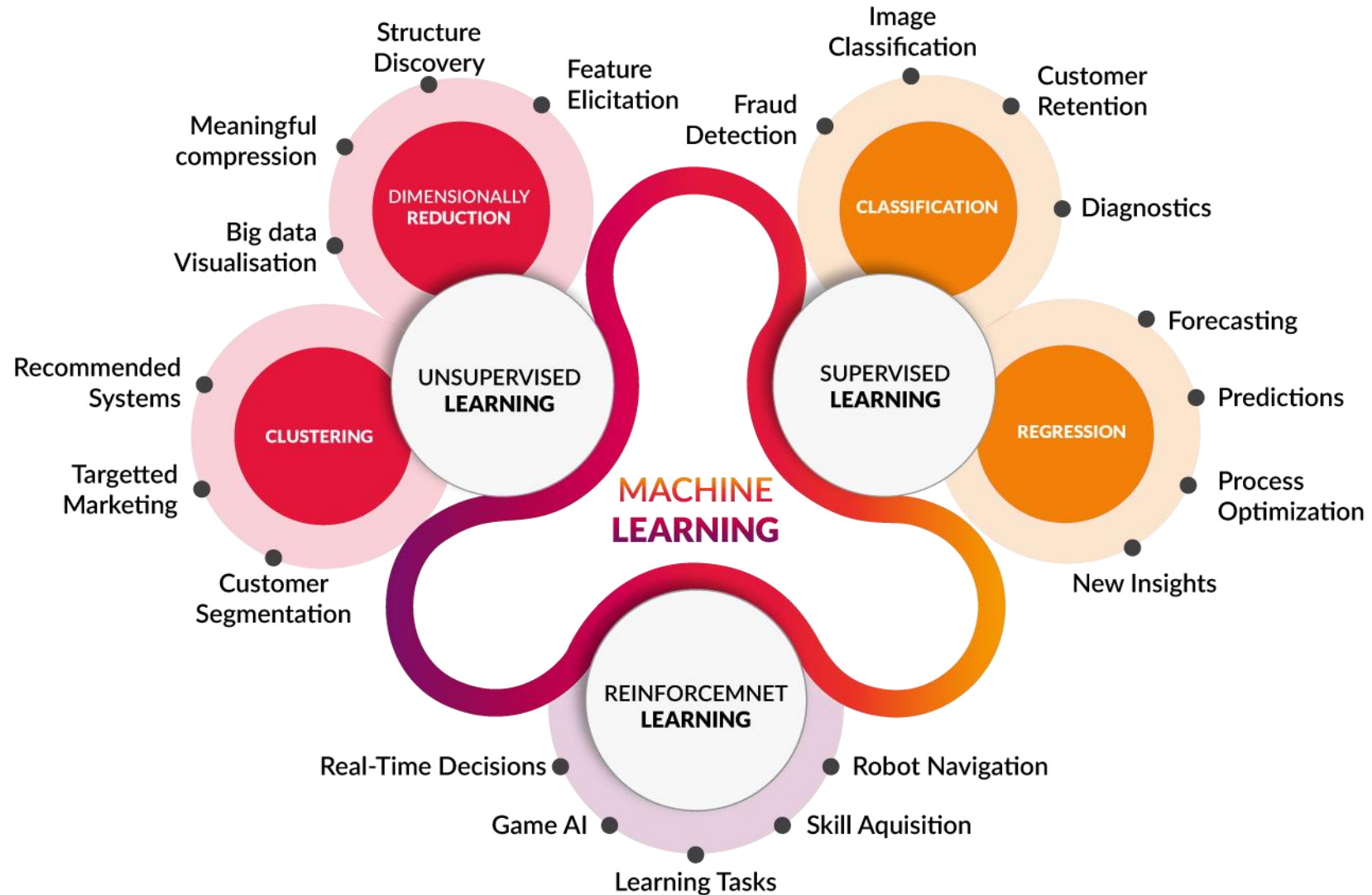


Since an early flush of optimism in the 1950s, smaller subsets of artificial intelligence – first machine learning, then deep learning, a subset of machine learning – have created ever larger disruptions.

Type of (Example-based) Learning



Type of learning



Type of learning

	supervised	unsupervised
continuous	Regression	Dimensionality Reduction
categorical	Classification	Clustering

Supervised Learning



Regression



What will be the temperature tomorrow?



Classification



Will it be hot or cold tomorrow?

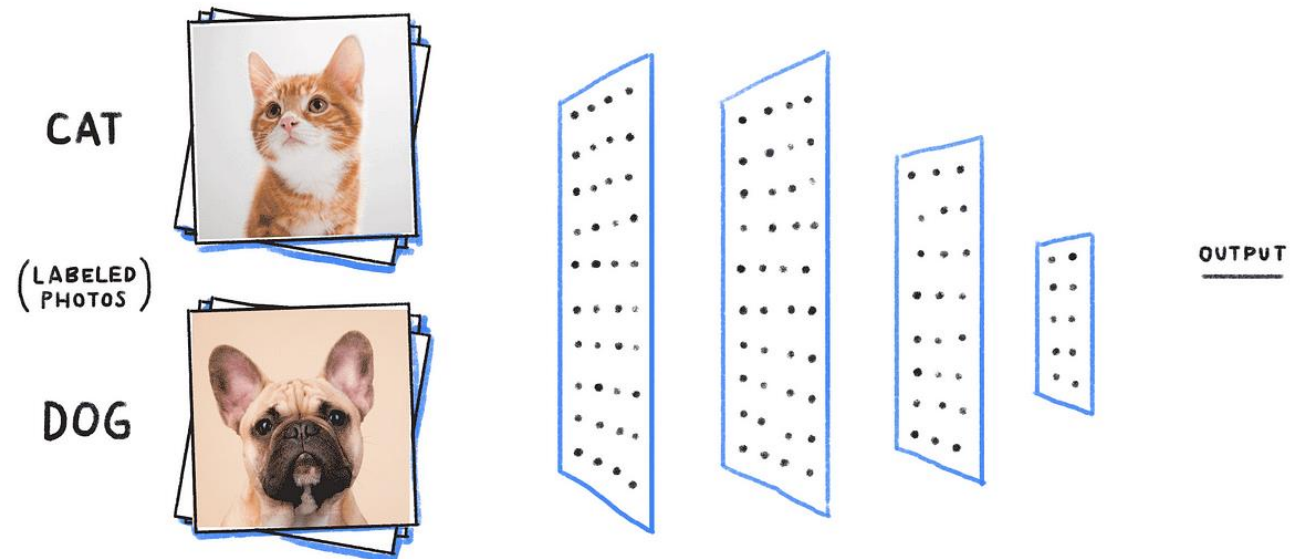


Supervised Learning

- Classification: We have labels

$$\mathcal{D} = \{(x_1, y_1), \dots, (x_k, y_k)\}$$

input (e.g., image) label



Supervised Learning

- Classification: We have labels

$$\mathcal{D} = \{(x_1, y_1), \dots, (x_k, y_k)\}$$

input (e.g., image) label

airplane



automobile



bird



cat



deer



dog



frog



horse



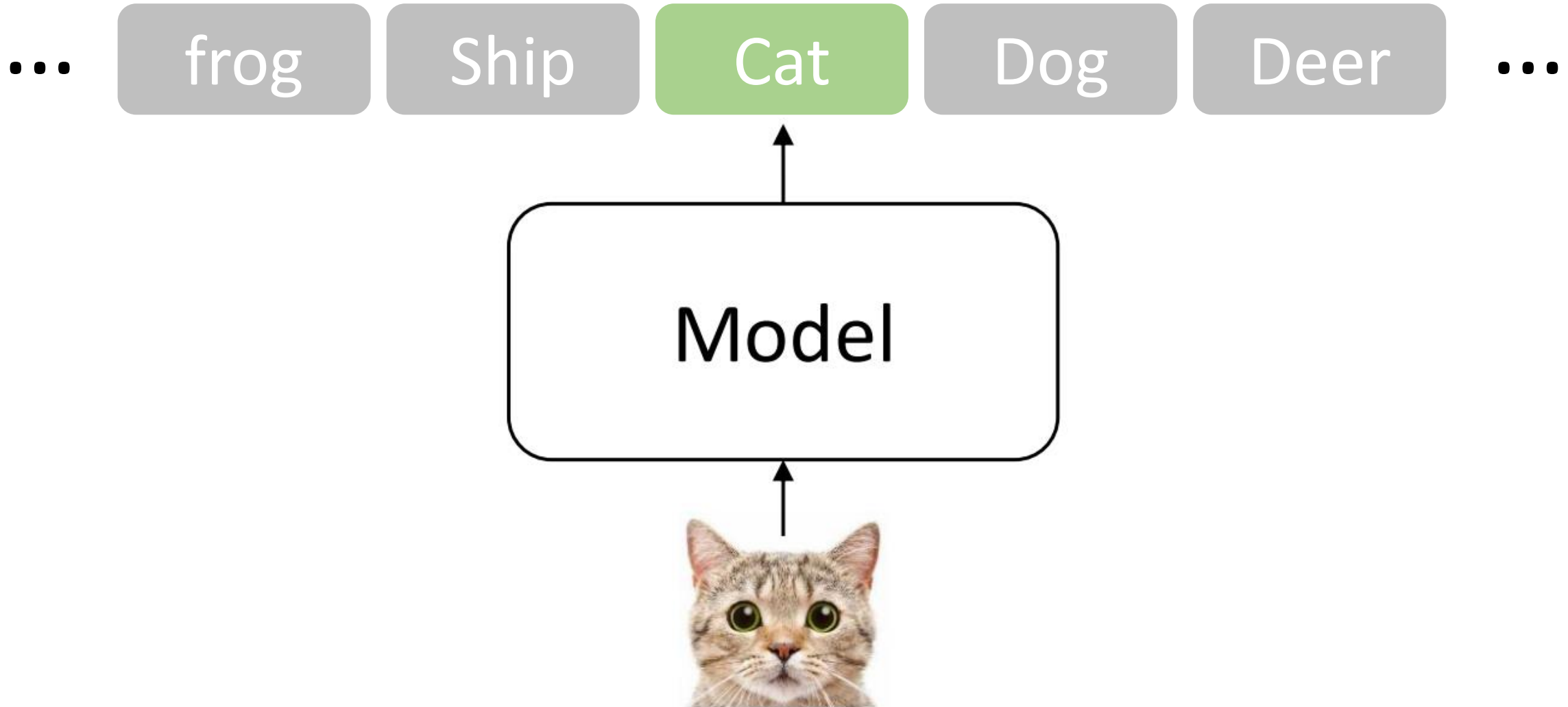
ship



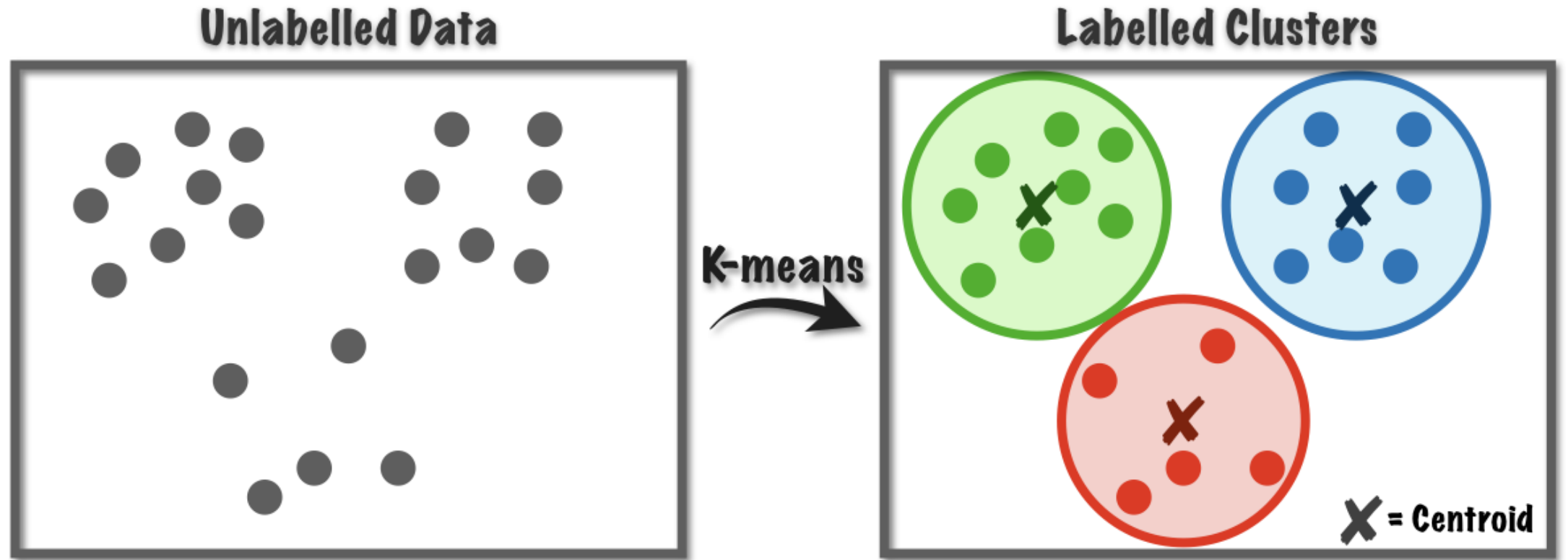
truck



Supervised Learning



Unsupervised Learning



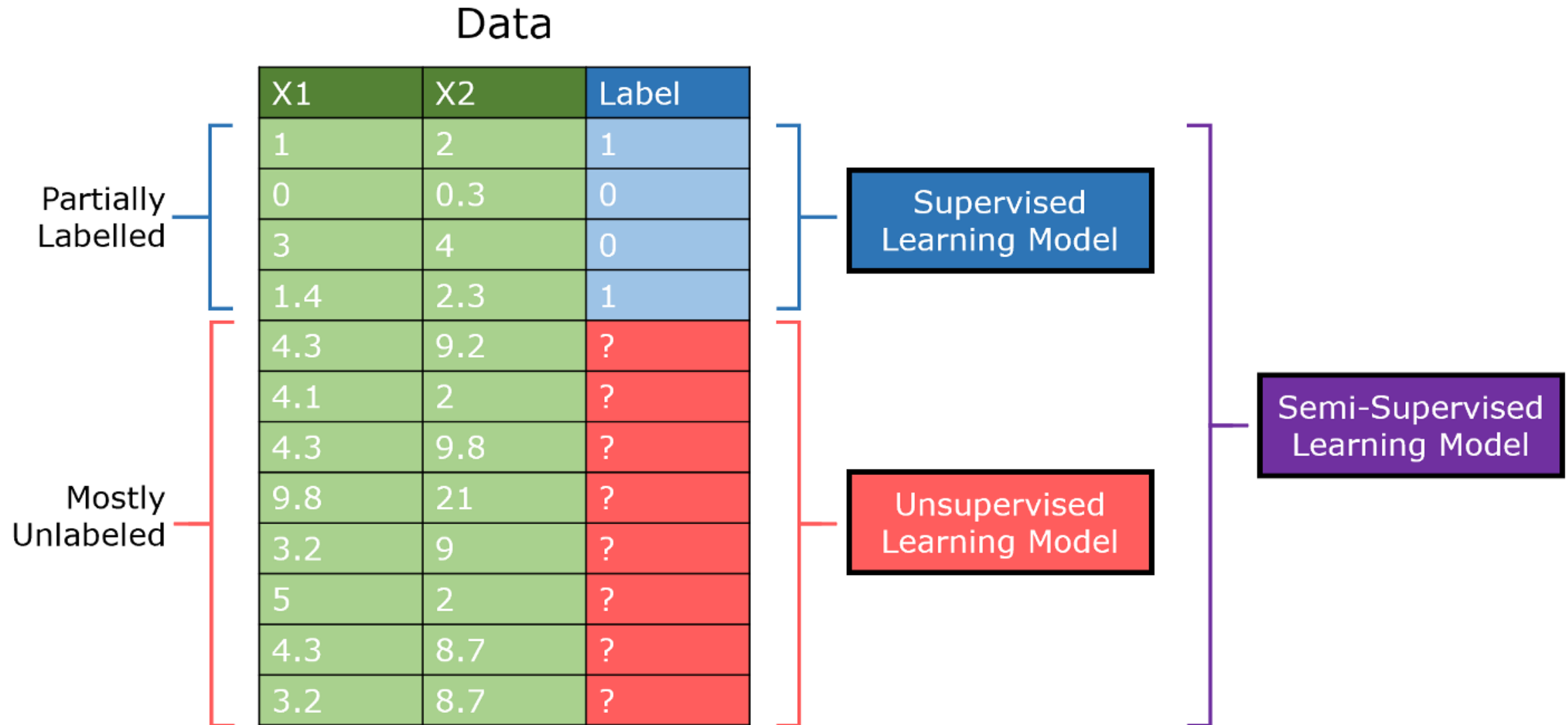
Unsupervised Learning



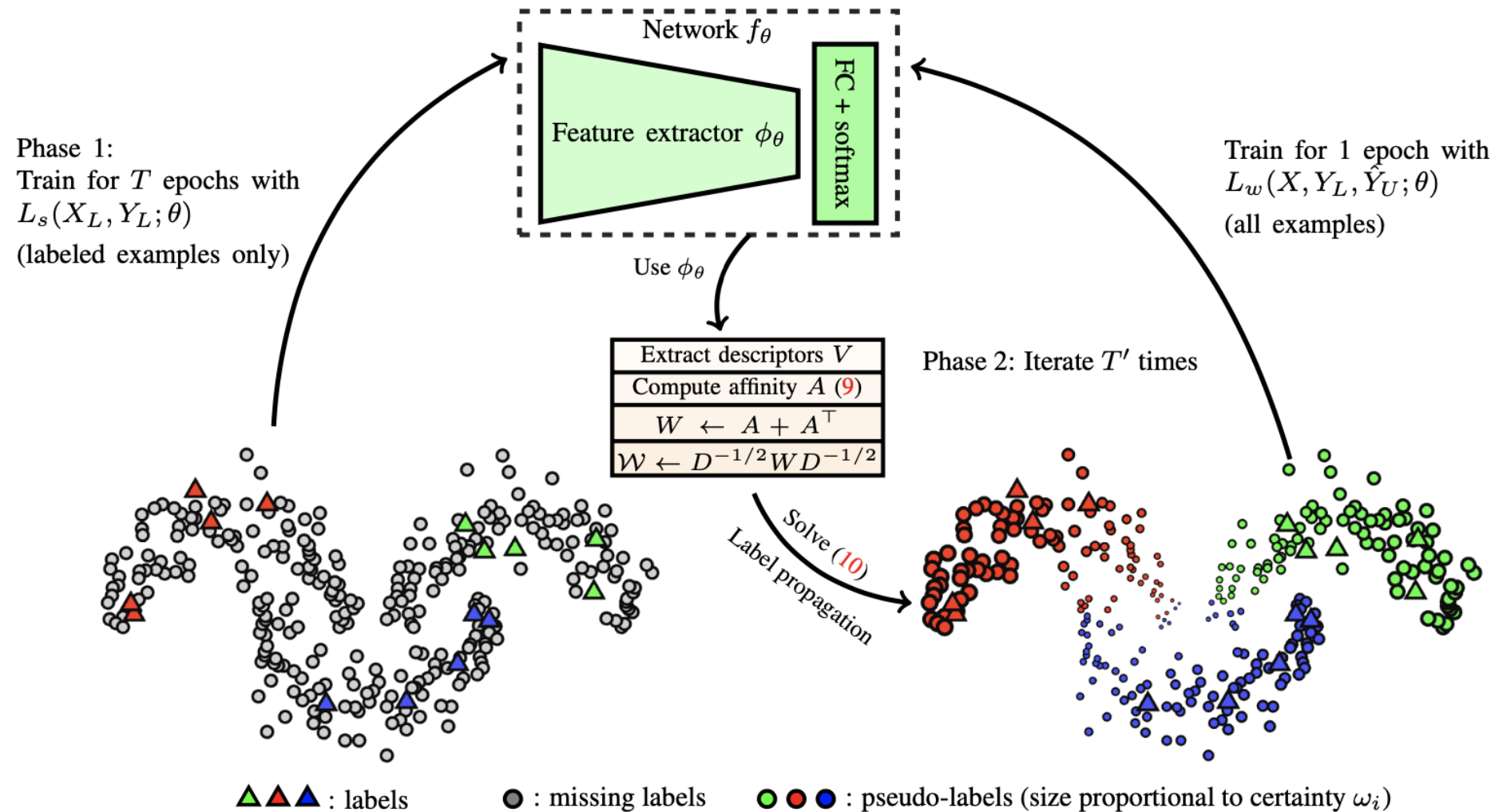
Classification

Clustering

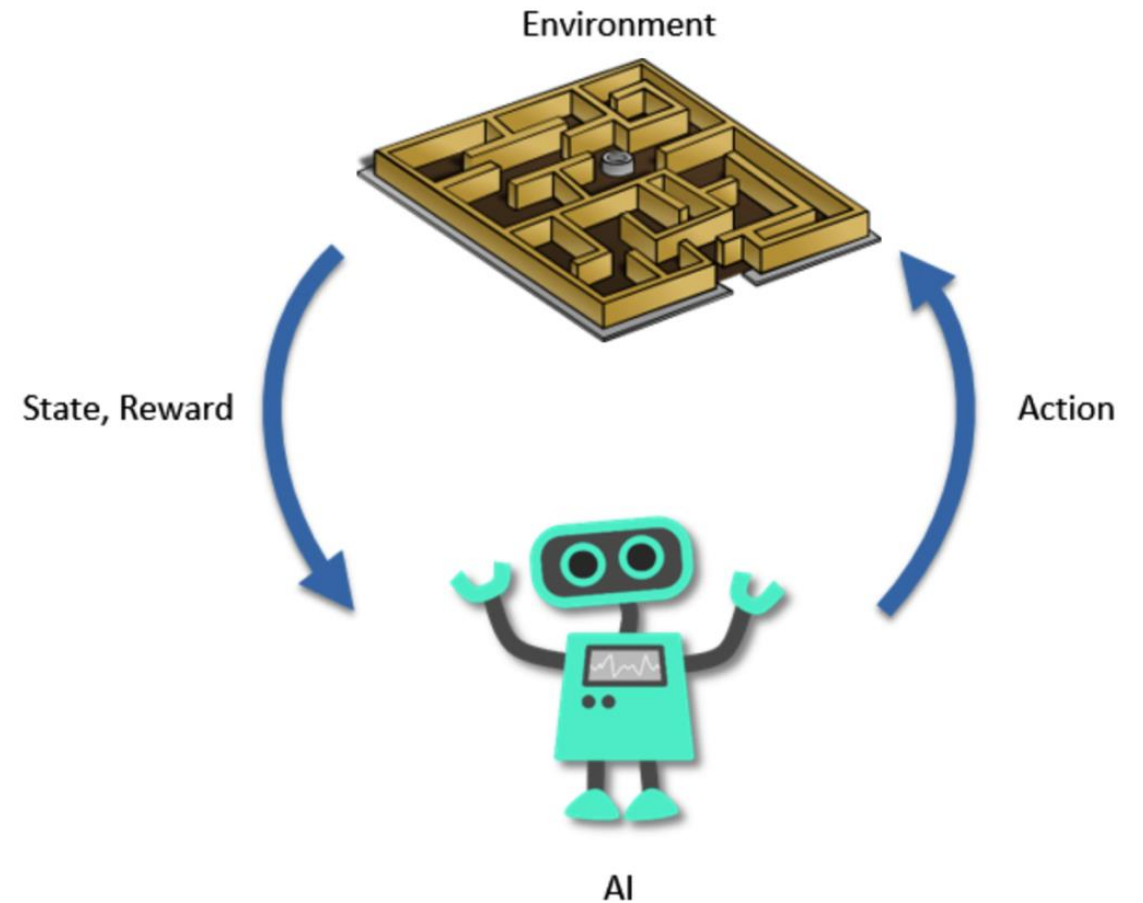
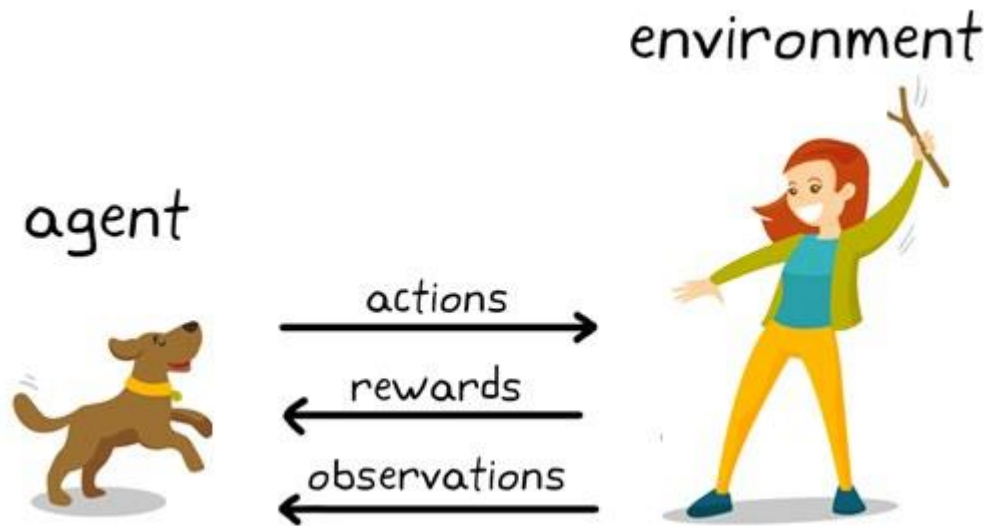
Semi Supervised Learning



Semi Supervised Learning

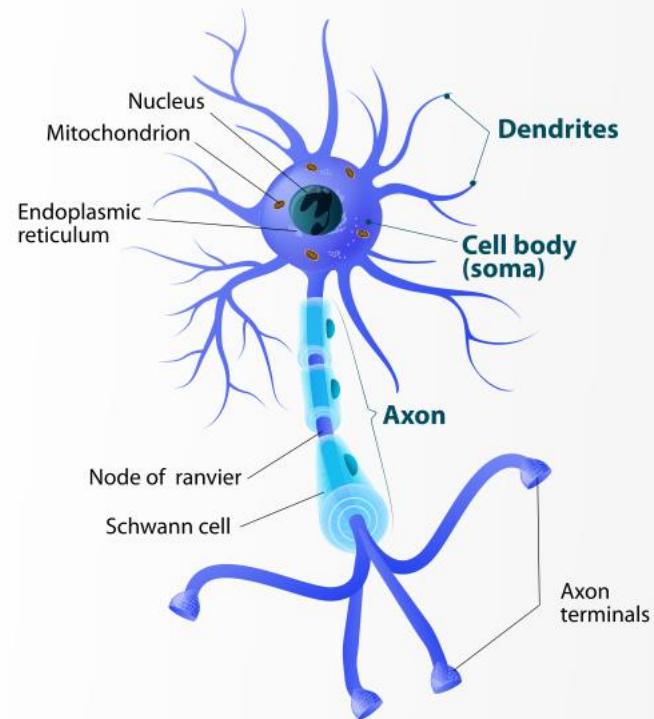


Reinforcement Learning

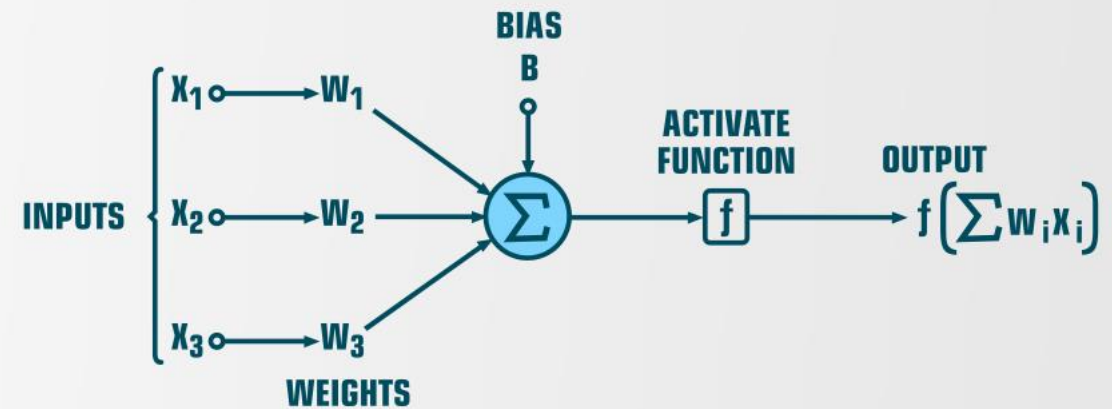


Basics

Structure of Typical Neuron

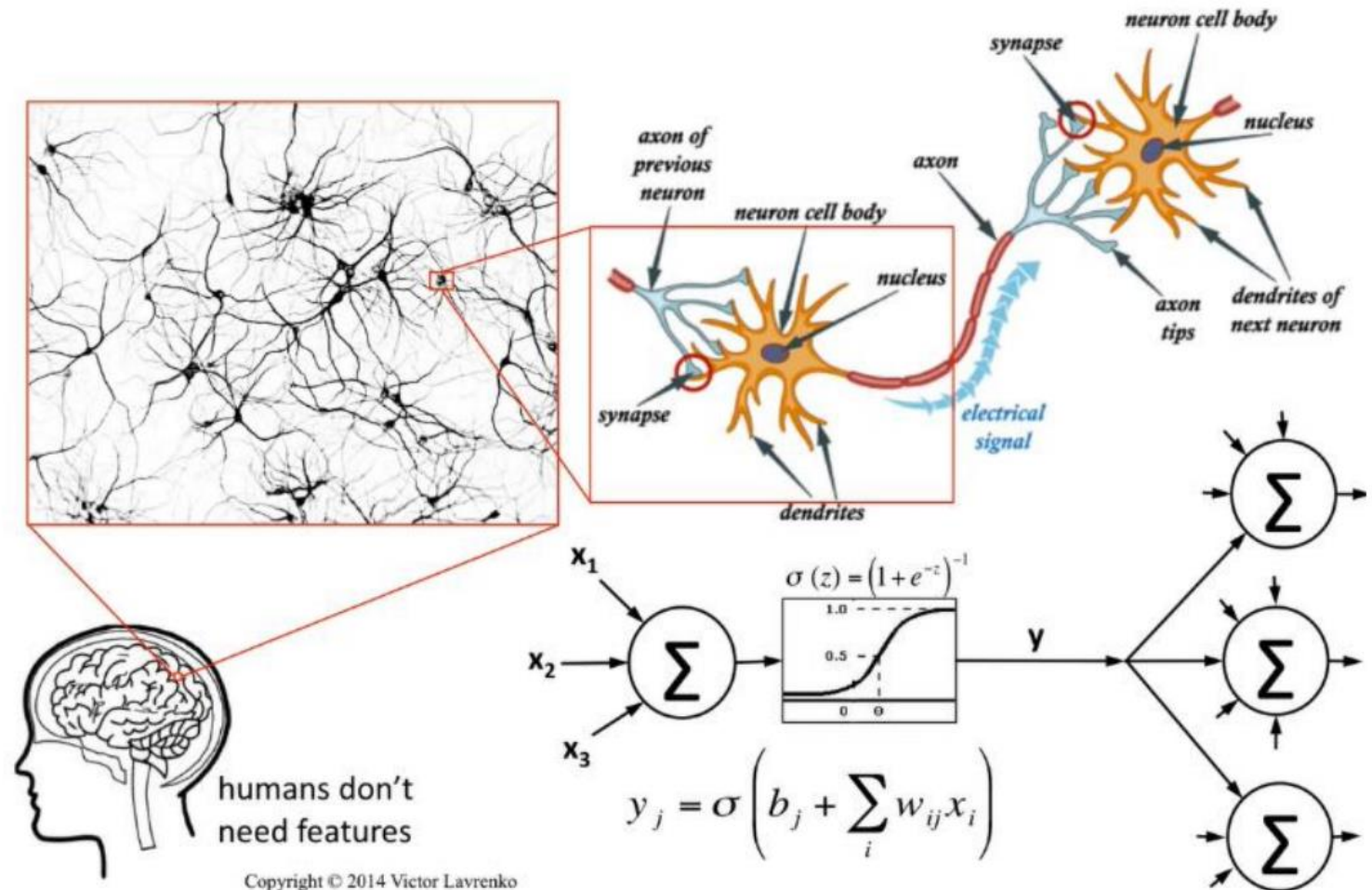


Structure of Artificial Neuron

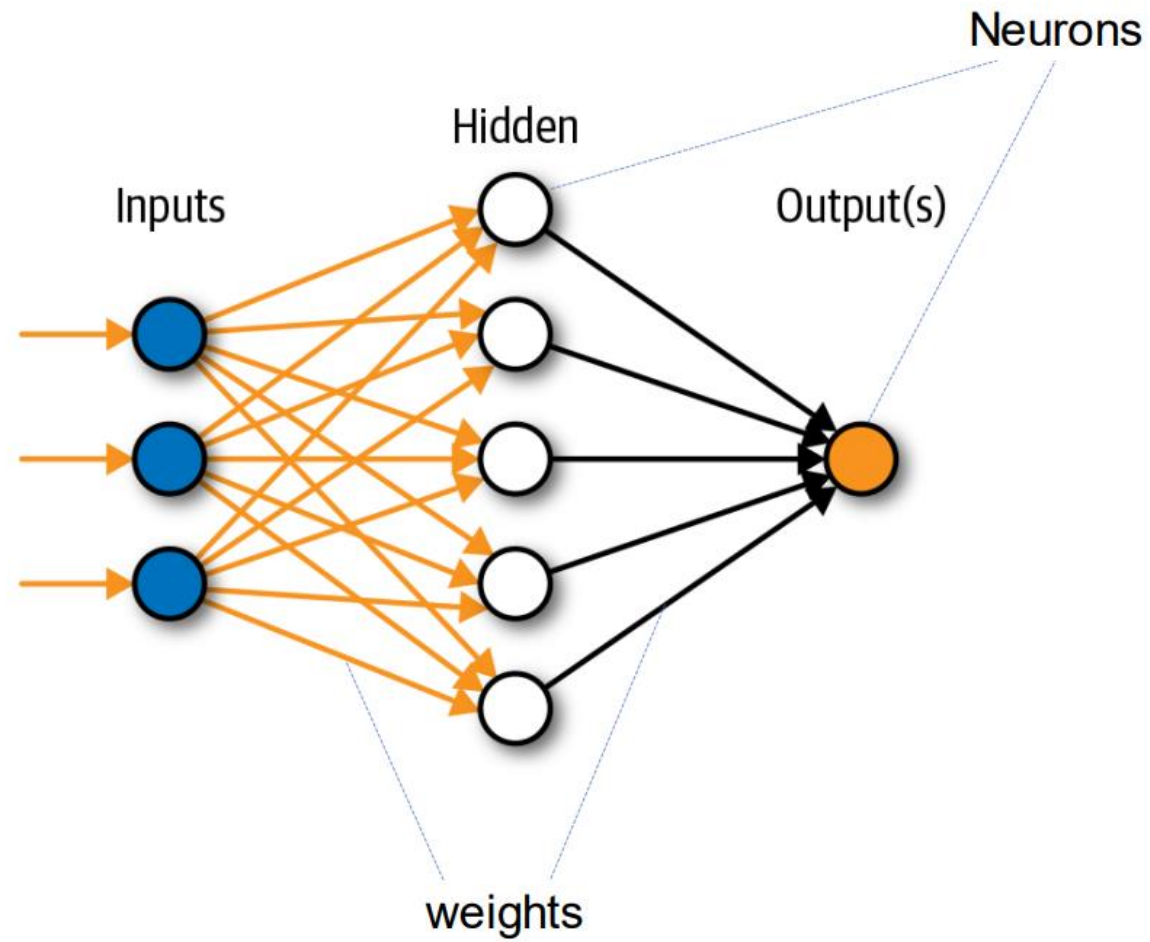


Basics

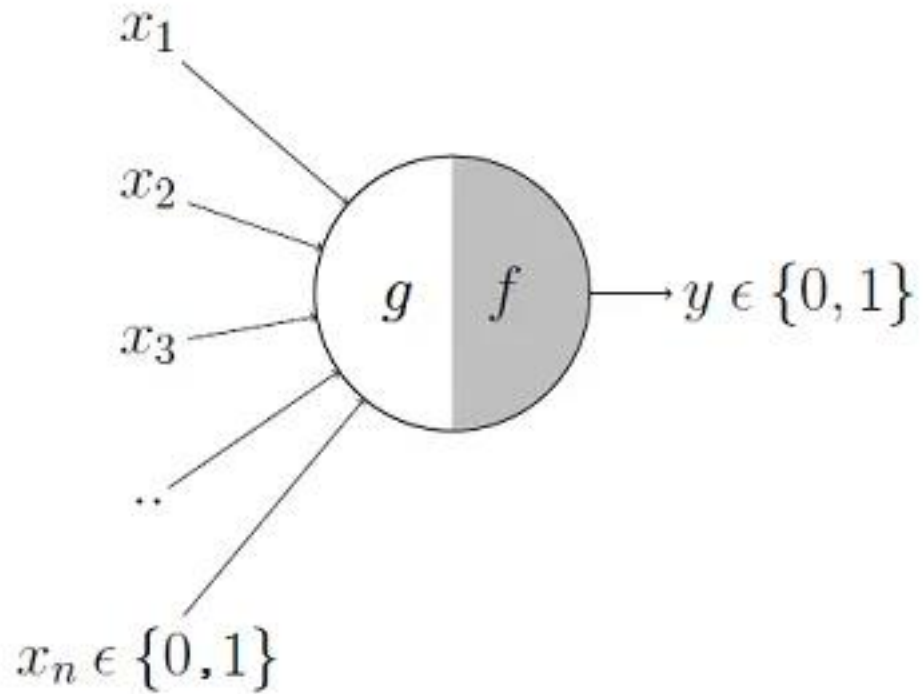
- Dendrites (inputs)
- Neurons (nodes)
- Synapses (weights)
- Axon (output)



Basics



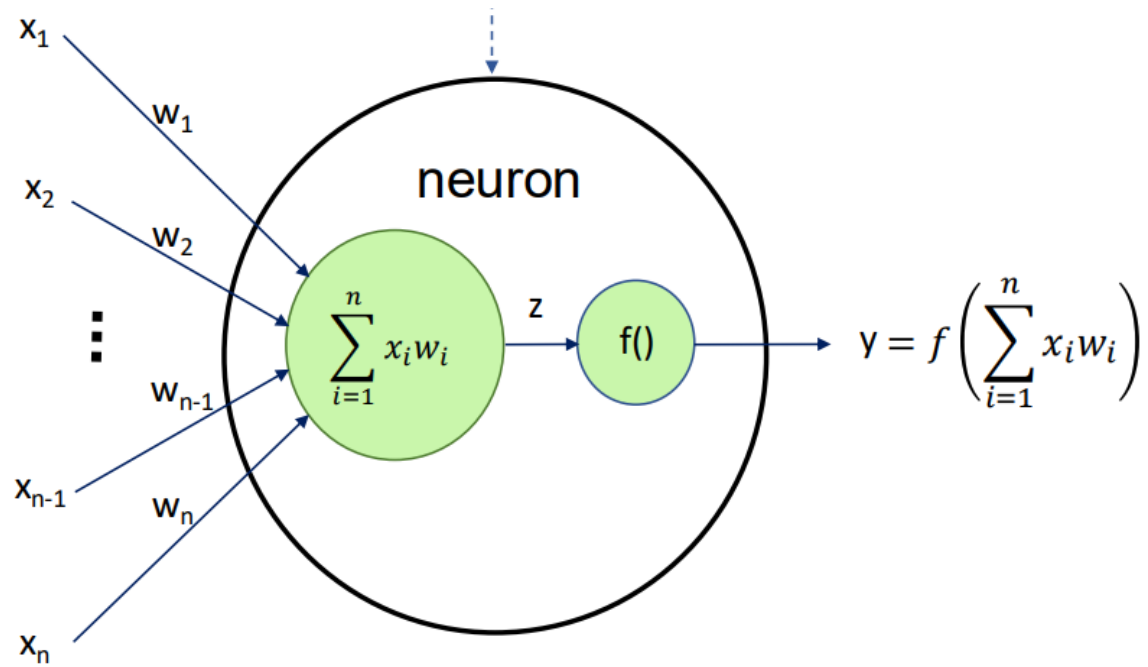
McCulloch-Pitts Neuron



$$g(x_1, x_2, x_3, \dots, x_n) = g(\mathbf{x}) = \sum_{i=1}^n x_i$$

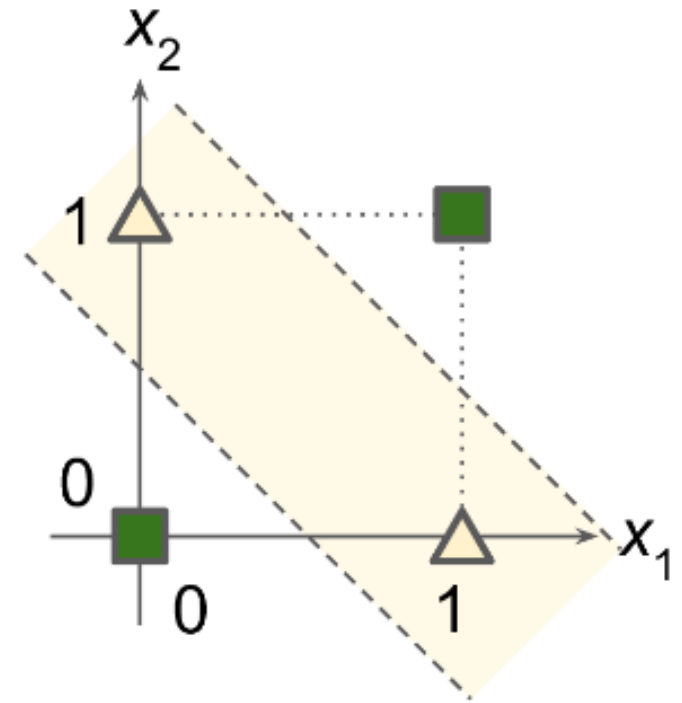
$$y = f(g(\mathbf{x})) = \begin{cases} 1 & \text{if } g(\mathbf{x}) \geq \theta \\ 0 & \text{if } g(\mathbf{x}) < \theta \end{cases}$$

Perceptron

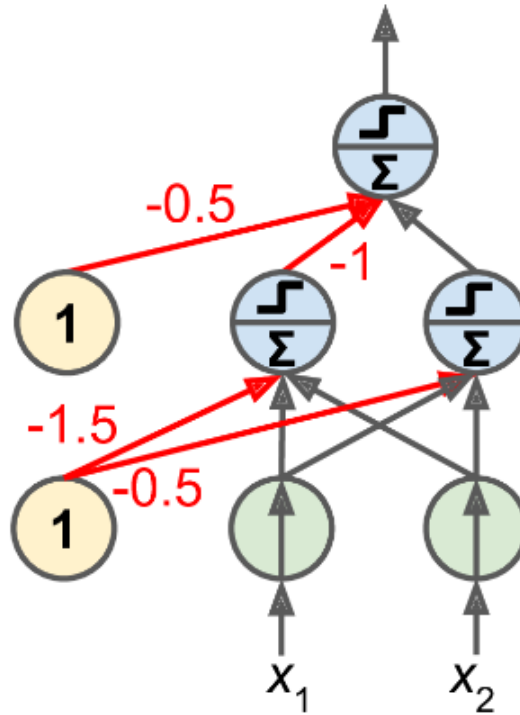
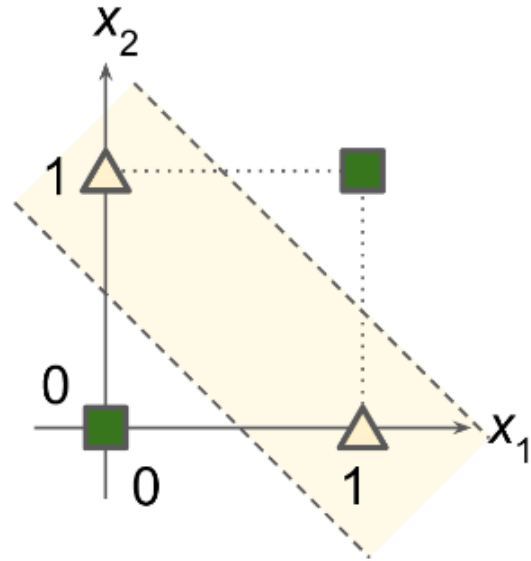


XOR problem

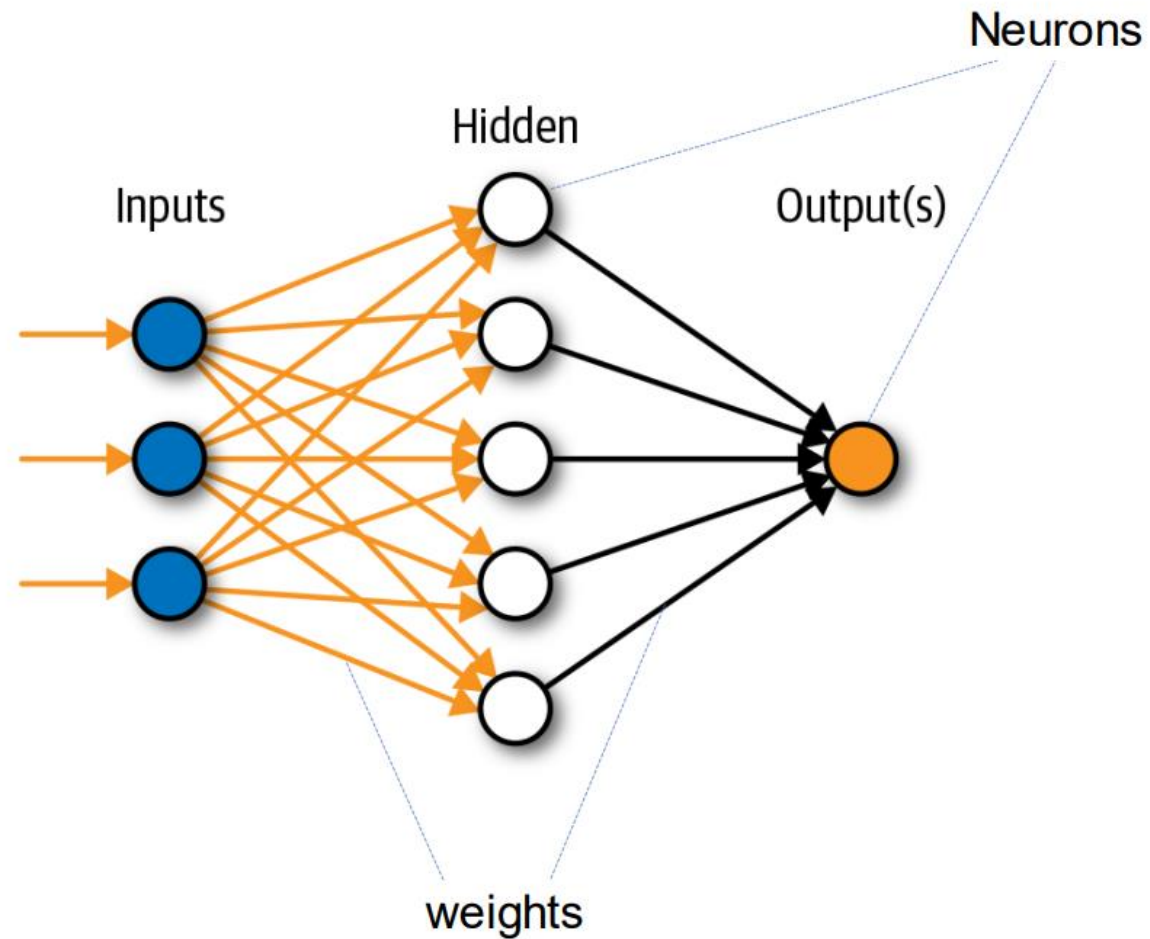
Perceptrons are incapable of solving some trivial problems



XOR problem



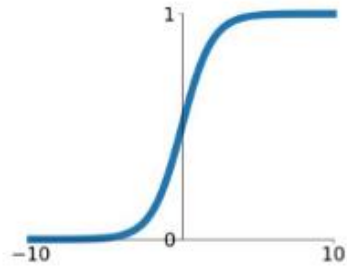
Multi-Layer Perceptron



Activation function

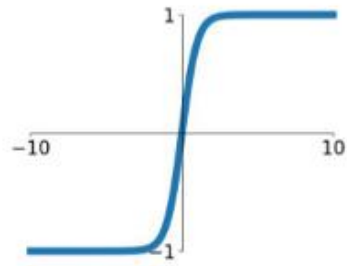
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



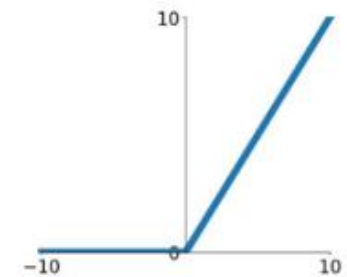
tanh

$$\tanh(x)$$



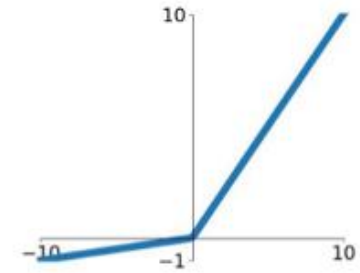
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

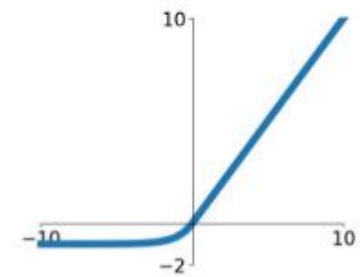


Maxout

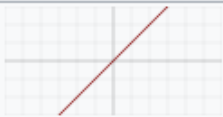
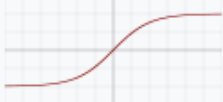
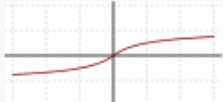
$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

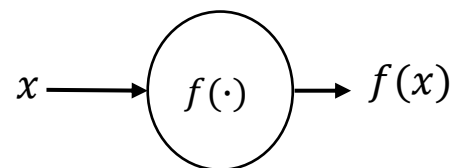
ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$

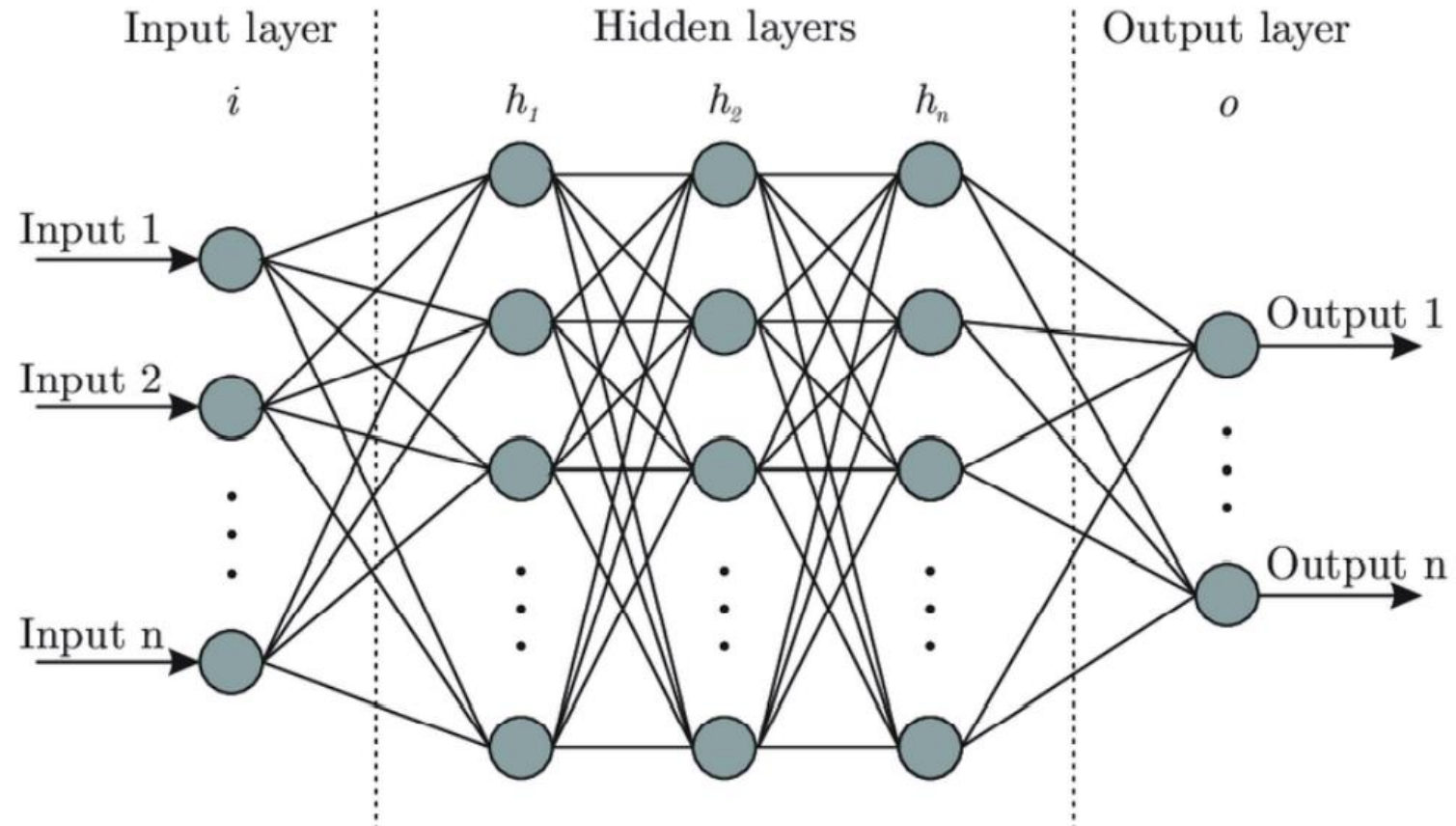


Activation function

Name	Plot	Equation	Derivative (with respect to x)	Range
Identity		$f(x) = x$	$f'(x) = 1$	$(-\infty, \infty)$
Binary step		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x \neq 0 \\ ? & \text{for } x = 0 \end{cases}$	$\{0, 1\}$
Logistic (a.k.a. Sigmoid or Soft step)		$f(x) = \sigma(x) = \frac{1}{1 + e^{-x}}$ ^[1]	$f'(x) = f(x)(1 - f(x))$	$(0, 1)$
TanH		$f(x) = \tanh(x) = \frac{(e^x - e^{-x})}{(e^x + e^{-x})}$	$f'(x) = 1 - f(x)^2$	$(-1, 1)$
ArcTan		$f(x) = \tan^{-1}(x)$	$f'(x) = \frac{1}{x^2 + 1}$	$(-\frac{\pi}{2}, \frac{\pi}{2})$
Softsign ^{[7][8]}		$f(x) = \frac{x}{1 + x }$	$f'(x) = \frac{1}{(1 + x)^2}$	$(-1, 1)$
Inverse square root unit (ISRU) ^[9]		$f(x) = \frac{x}{\sqrt{1 + \alpha x^2}}$	$f'(x) = \left(\frac{1}{\sqrt{1 + \alpha x^2}} \right)^3$	$(-\frac{1}{\sqrt{\alpha}}, \frac{1}{\sqrt{\alpha}})$
Rectified linear unit (ReLU) ^[10]		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$[0, \infty)$



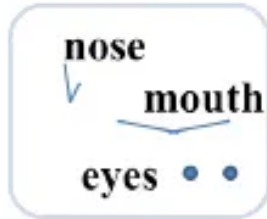
Layer



Why we need layers?



Layer 1: detect edges & corners



Layer 2: form feature groups



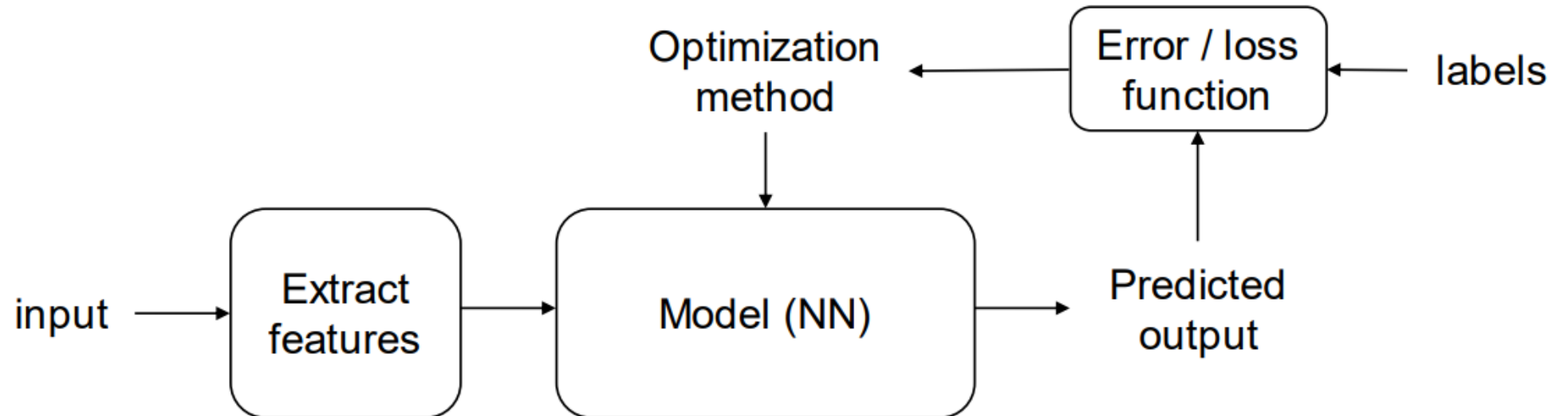
**Layer 3: detect high level
objects, faces, etc.**

Learning Process

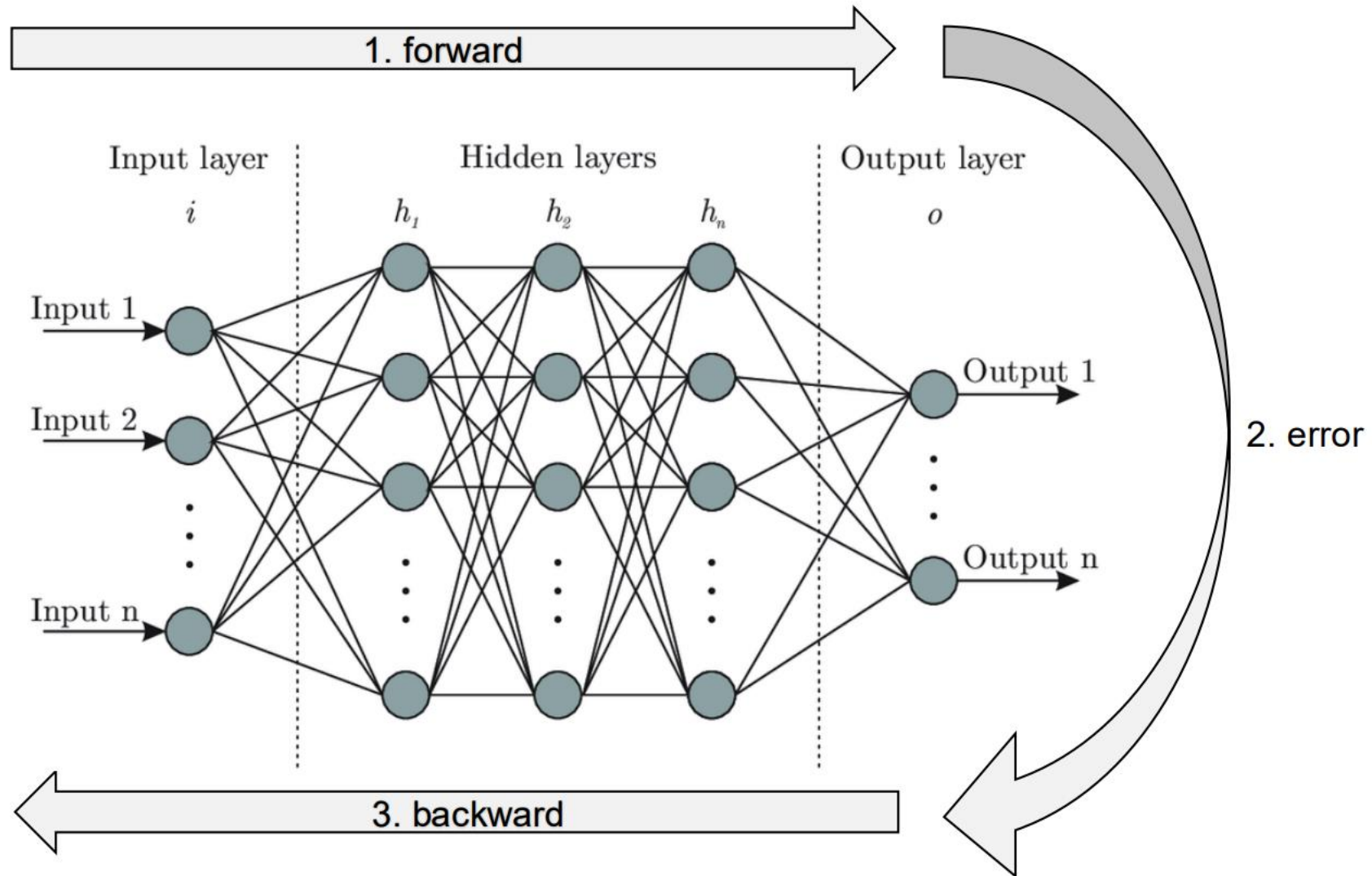
$$\mathcal{D} = \{(x_1, y_1), \dots, (x_k, y_k)\}$$

input (e.g., image) label

- Here the notations are based on supervised learning
- Train a model



Train a neural network



Error

- Compare network output ($\hat{y}$) with label (y)

Regression losses

$$MSE = \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{n}$$

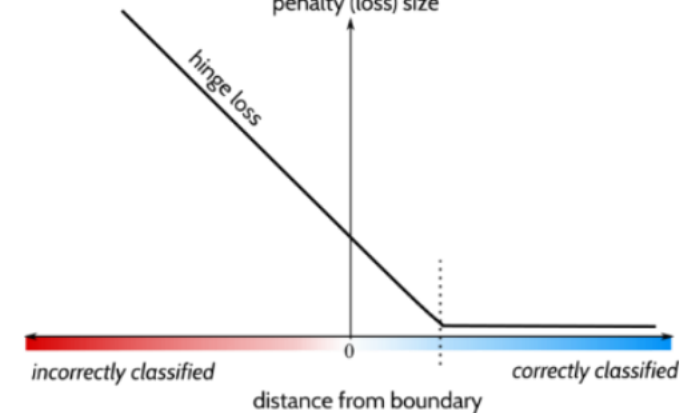
$$MAE = \frac{\sum_{i=1}^n |y_i - \hat{y}_i|}{n}$$

$$MBE = \frac{\sum_{i=1}^n (y_i - \hat{y}_i)}{n}$$

$$D_{KL}(P \parallel Q) = - \sum_{x \in \mathcal{X}} P(x) \log \left(\frac{Q(x)}{P(x)} \right)$$

Classification losses

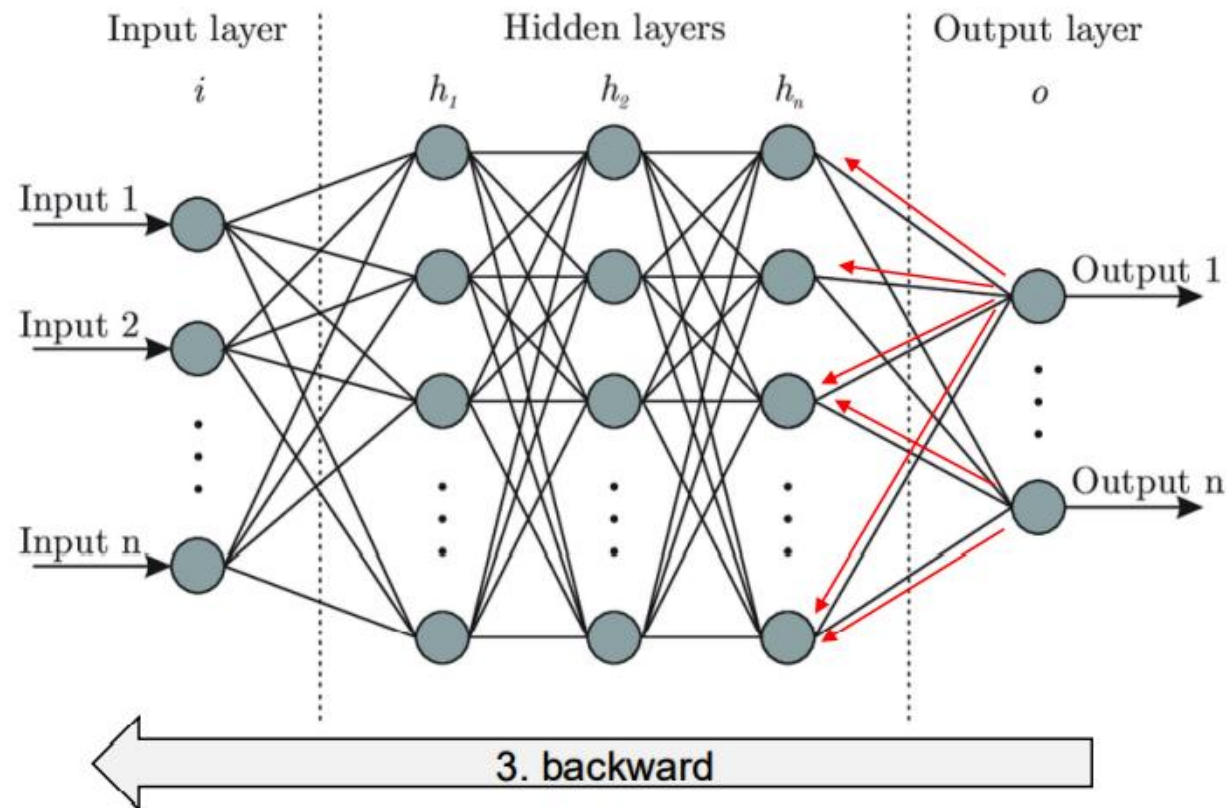
$$SVM Loss = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$



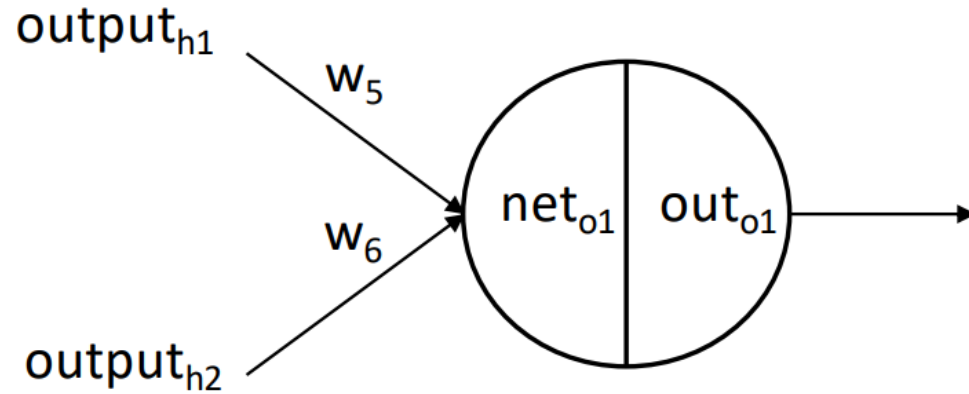
$$CrossEntropyLoss = -(y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i))$$

Backward

- Error Backpropagation
 - Compute the derivative of error with respect to weight



Backward



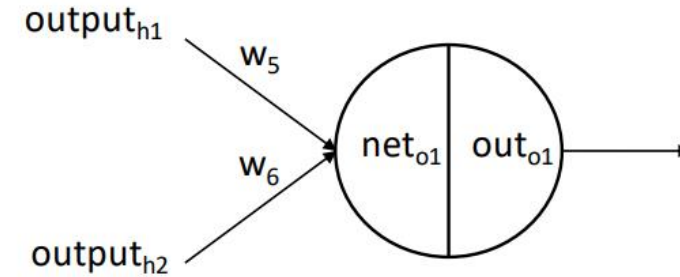
$$E_{total} = E_{o1} + E_{o2}$$

$$E_{o1} = \frac{1}{2}(target_{o1} - out_{o1})^2$$

$$\frac{\partial net_{o1}}{\partial w_5} * \frac{\partial out_{o1}}{\partial net_{o1}} * \frac{\partial E_{total}}{\partial out_{o1}} = \frac{\partial E_{total}}{\partial w_5}$$

Backward

- Error Backpropagation



$$\begin{aligned} & net_{o1} = w_5 * out_{h1} + w_6 * out_{h2} + b_2 * 1 \\ & out_{o1} = \frac{1}{1 + e^{-net_{o1}}} \\ & E_{total} = \frac{1}{2} (target_{o1} - out_{o1})^2 + \frac{1}{2} (target_{o2} - out_{o2})^2 \\ & \frac{\partial net_{o1}}{\partial w_5} * \frac{\partial out_{o1}}{\partial net_{o1}} * \frac{\partial E_{total}}{\partial out_{o1}} = \frac{\partial E_{total}}{\partial w_5} \\ & \frac{\partial net_{o1}}{\partial w_5} = out_{h1} \\ & \frac{\partial out_{o1}}{\partial net_{o1}} = out_{o1} (1 - out_{o1}) \\ & \frac{\partial E_{total}}{\partial out_{o1}} = -(target_{o1} - out_{o1}) \end{aligned}$$

Backward

Backward

- How to Update weights with $\frac{\partial E_{total}}{\partial w_i}$?
- Gradient Descent

The diagram shows the weight update equation $W_i^t = W_i^{t-1} - a \left(\frac{\partial E_{total}}{\partial w_5} \right)$. Four blue arrows point to the components of the equation: an upward arrow from 'New weight' to W_i^t , a downward arrow from 'Old weight' to W_i^{t-1} , an upward arrow from 'Learning rate' to a , and a downward arrow from 'Derivative of error with respect to weight' to $\left(\frac{\partial E_{total}}{\partial w_5} \right)$.

$$W_i^t = W_i^{t-1} - a \left(\frac{\partial E_{total}}{\partial w_5} \right)$$

New weight

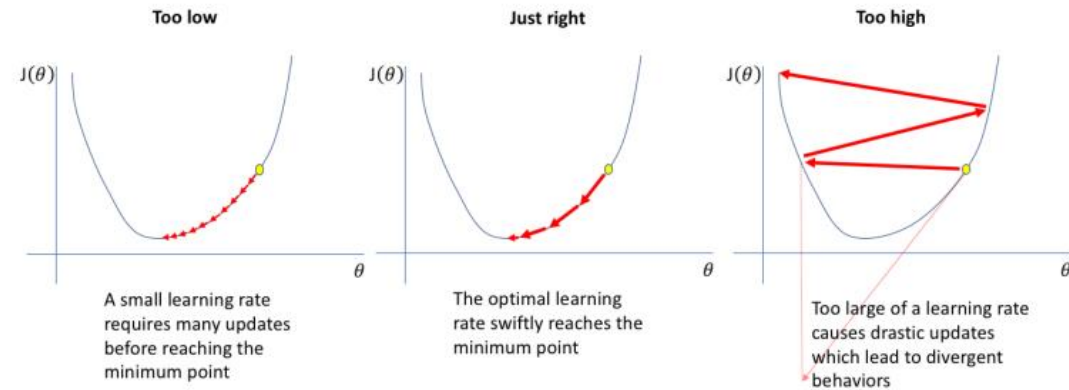
Old weight

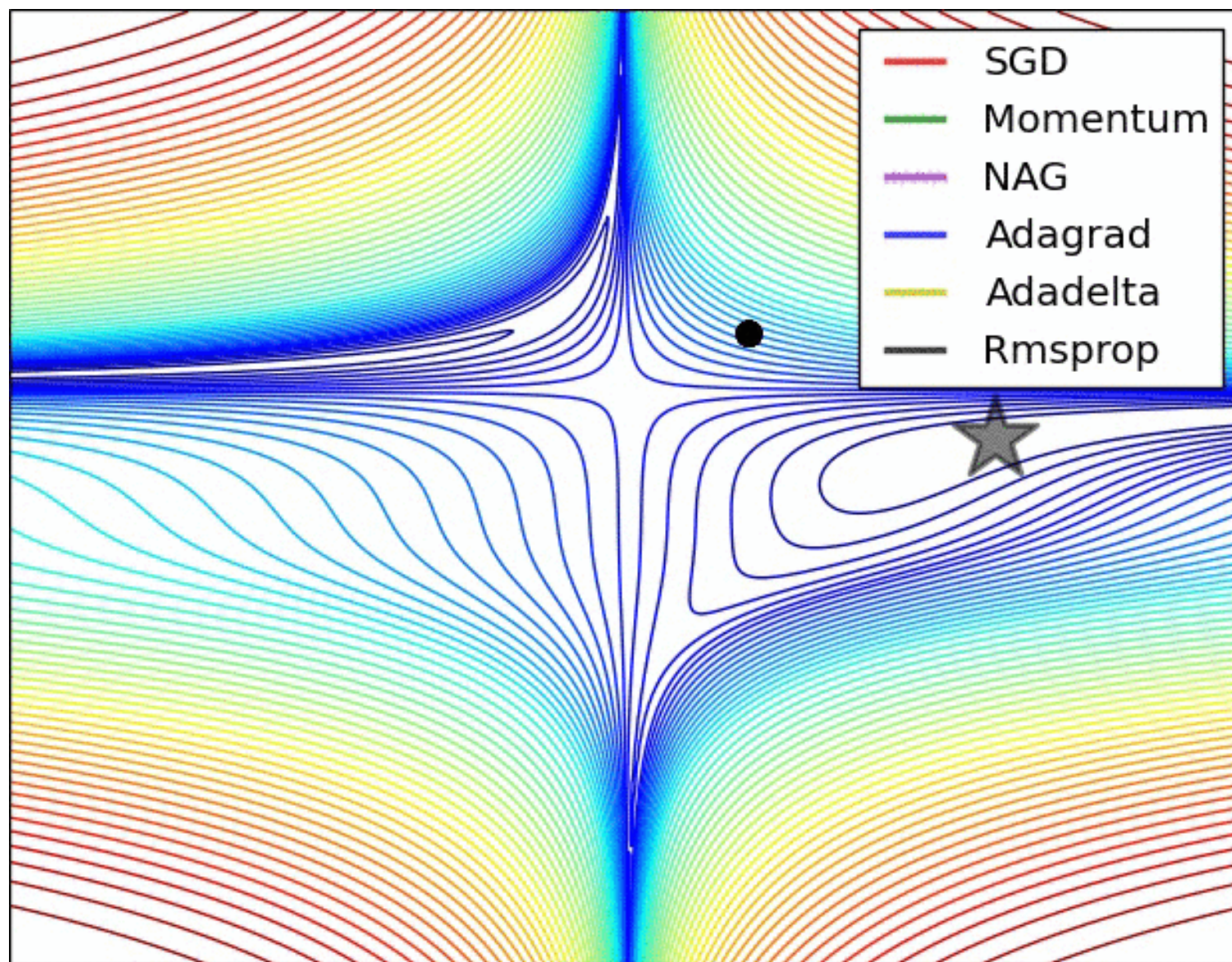
Derivative of error with respect to weight

Learning rate

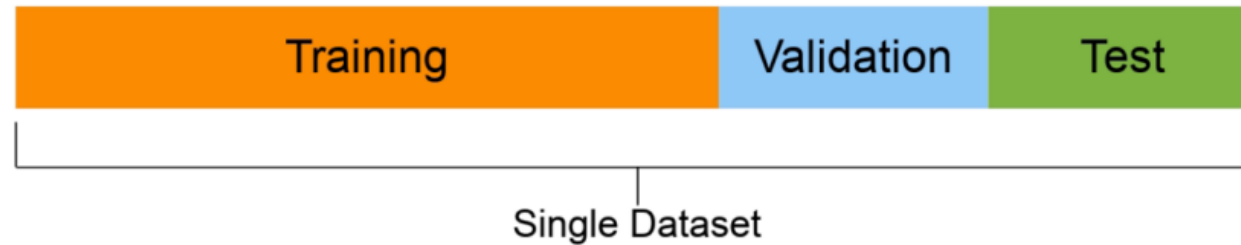
Optimization methods

- Stochastic Gradient Descent
- Mini Batch Gradient Descent
- Momentum
- Nesterov accelerated gradient
- Adagrad
- AdaDelta
- RMSProp
- Adam





How to split our data?



- A training dataset is a dataset of examples used during the learning process and is used to fit the **parameters** (e.g., weights) of, for example, a classifier.
- A validation dataset is a dataset of examples used to tune the **hyperparameters** (i.e. the architecture) of a classifier.
- A test dataset is a dataset that is independent of the training dataset, but that follows the same probability distribution as the training dataset. If a model fit to the training dataset also fits the test dataset well, minimal overfitting has taken place. A better fitting of the training dataset as opposed to the test dataset usually points to overfitting.

How to split our data?

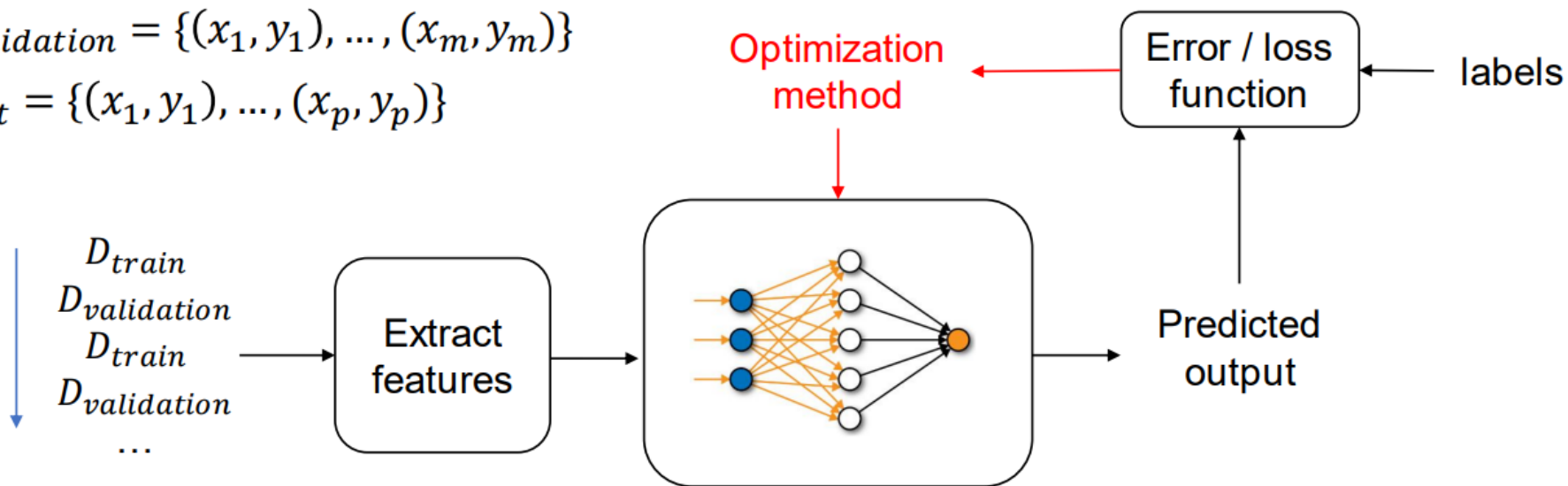
- Divide the data to three part

$$D = D_{train} + D_{validation} + D_{test}$$

$$D_{train} = \{(x_1, y_1), \dots, (x_n, y_n)\}$$

$$D_{validation} = \{(x_1, y_1), \dots, (x_m, y_m)\}$$

$$D_{test} = \{(x_1, y_1), \dots, (x_p, y_p)\}$$



Epoch and Batch

- Batch: Number of data which error is computed based on them in one forward step.
 - Batch Gradient Descent: Batch Size = Size of Training Set
 - Stochastic Gradient Descent: Batch Size = 1
 - Mini-Batch Gradient Descent: $1 < \text{Batch Size} < \text{Size of Training Set}$
- Epoch: One Epoch is when an entire dataset is passed forward and backward through the neural network only once.

Example

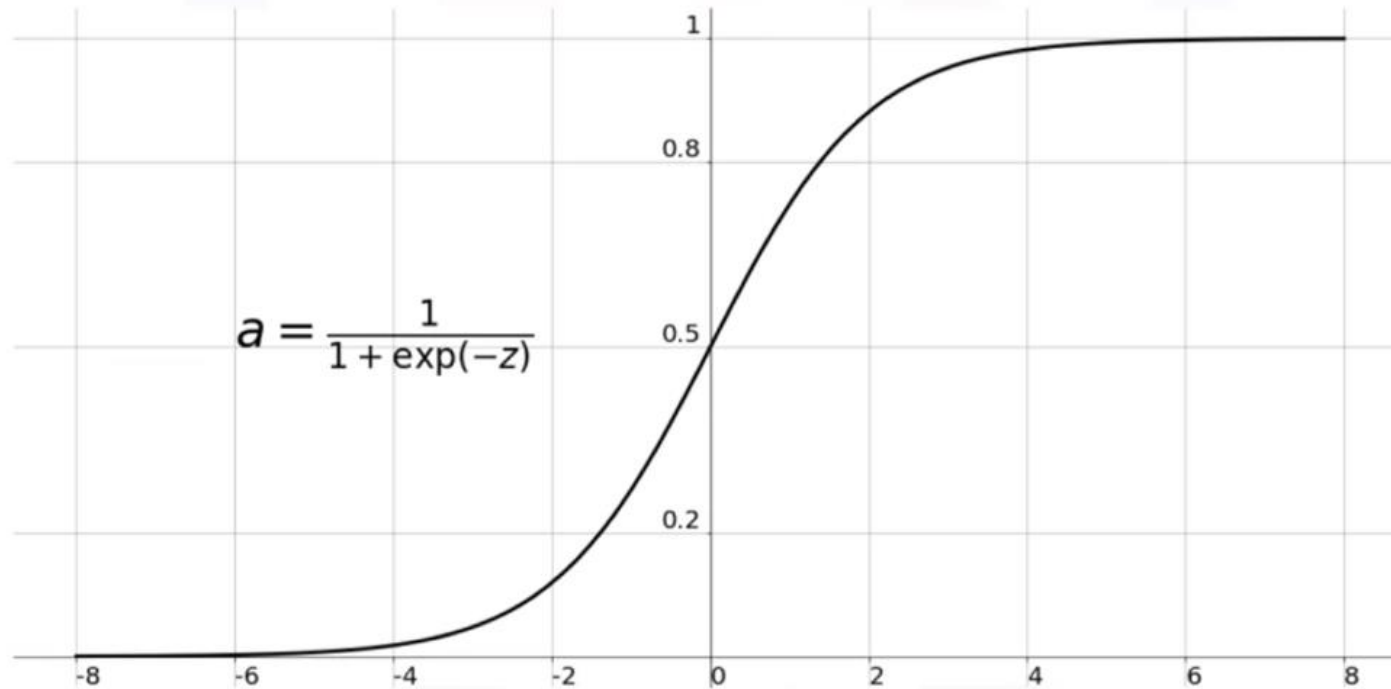


Size of training = 200, Epoch = 10, Batch Size = 5

- How many batches we have?
- How many times backward step is done?

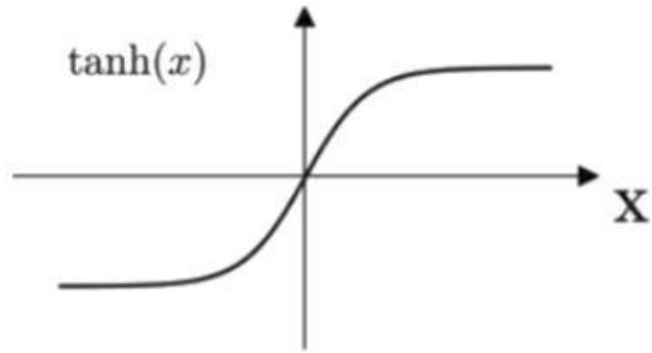
Activation function

Sigmoid Function

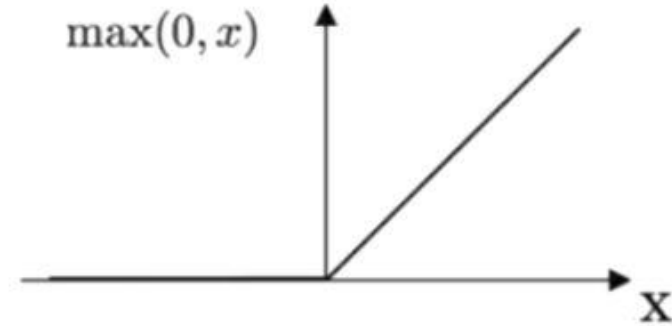


Activation function

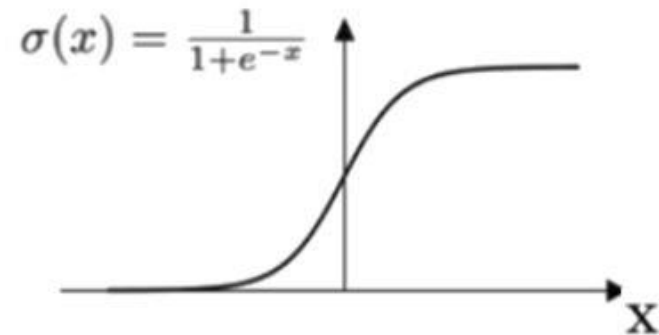
Tanh



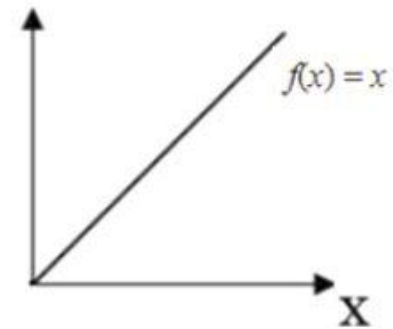
ReLU



Sigmoid



Linear



Sigmoid or Tanh?

- The hyperbolic tangent activation function typically performs better than the logistic sigmoid
- Both the sigmoid and Tanh functions can make the model more susceptible to problems during training, via the so-called vanishing gradients problem

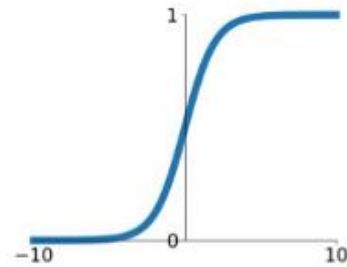
Choosing activation function

Hidden Layer:

The activation function used in hidden layers is typically chosen based on the type of neural network architecture.

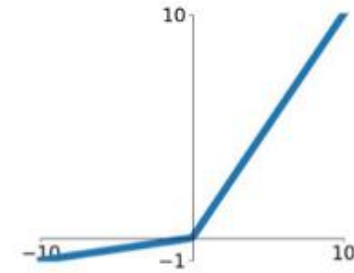
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



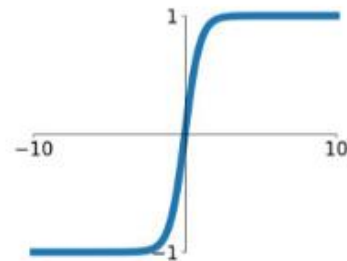
Leaky ReLU

$$\max(0.1x, x)$$



tanh

$$\tanh(x)$$

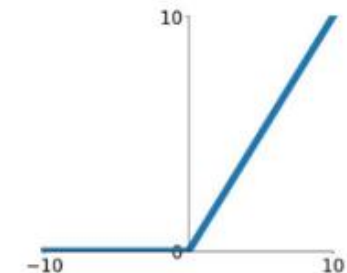


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

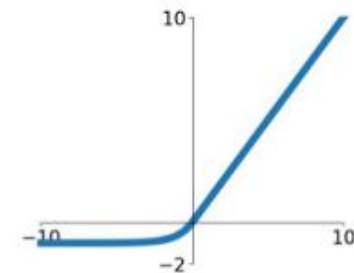
ReLU

$$\max(0, x)$$



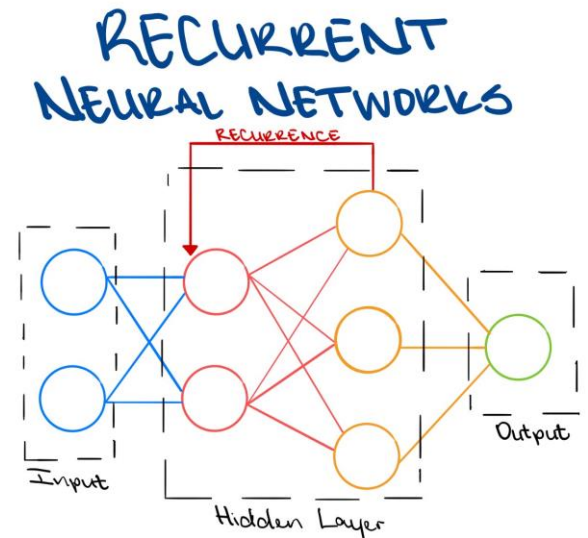
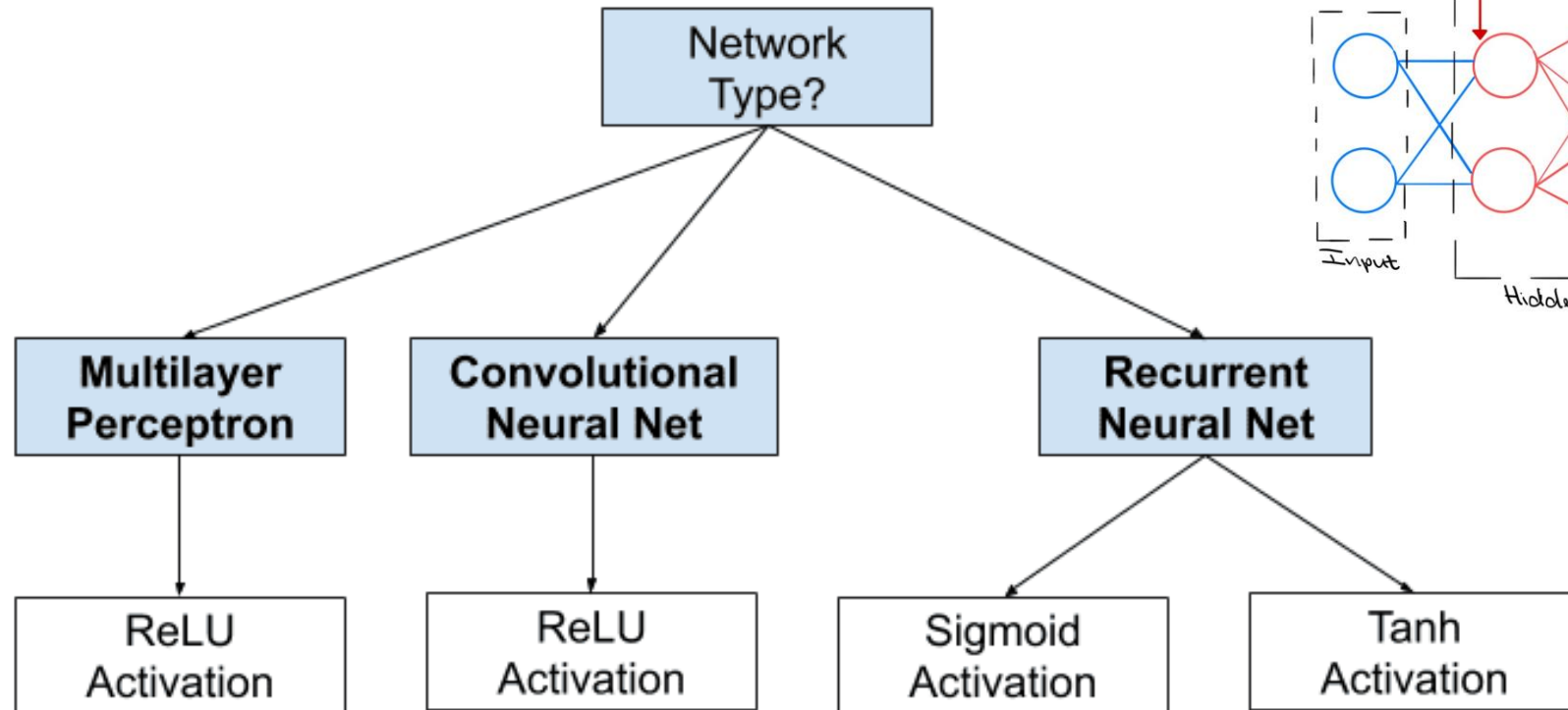
ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



Choosing activation function

How to Choose an Hidden Layer Activation Function



Multilabel Classification —> Sigmoid

Classification

hidden

hidden

logits

apple: yes/no?

bear: yes/no?

candy: yes/no?

dog: yes/no?

egg: yes/no?

Softmax

Output
layer

$$\begin{bmatrix} 1.3 \\ 5.1 \\ 2.2 \\ 0.7 \\ 1.1 \end{bmatrix}$$

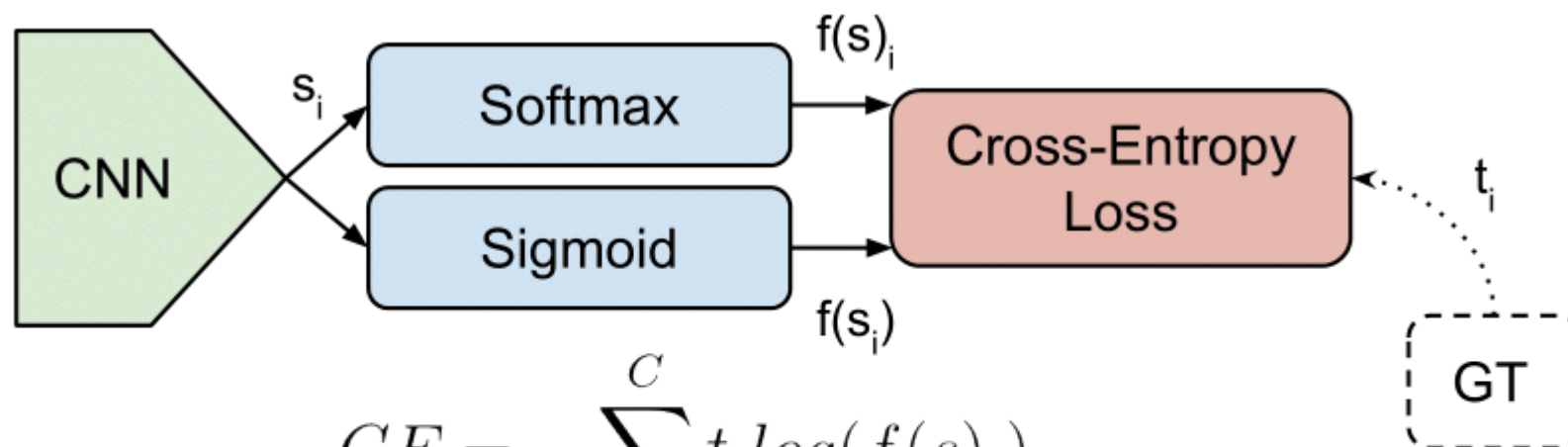
Softmax
activation function

$$\frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

Probabilities

$$\begin{bmatrix} 0.02 \\ 0.90 \\ 0.05 \\ 0.01 \\ 0.02 \end{bmatrix}$$

Binary Cross Entropy



$$CE = - \sum_i^C t_i \log(f(s)_i)$$

$$CE = - \sum_{i=1}^{C'=2} t_i \log(f(s_i)) = -t_1 \log(f(s_1)) - (1 - t_1) \log(1 - f(s_1))$$

Multiclass cross-entropy

Indicator variable

Prob of class j

$$-\sum_{j=1}^M y_j \log(p(y_j))$$

Sum over trials

Sum over classes

$$-\sum_{i=1}^N y_i \log(p(y_i)) + (1 - y_i) \log(1 - p(y_i))$$

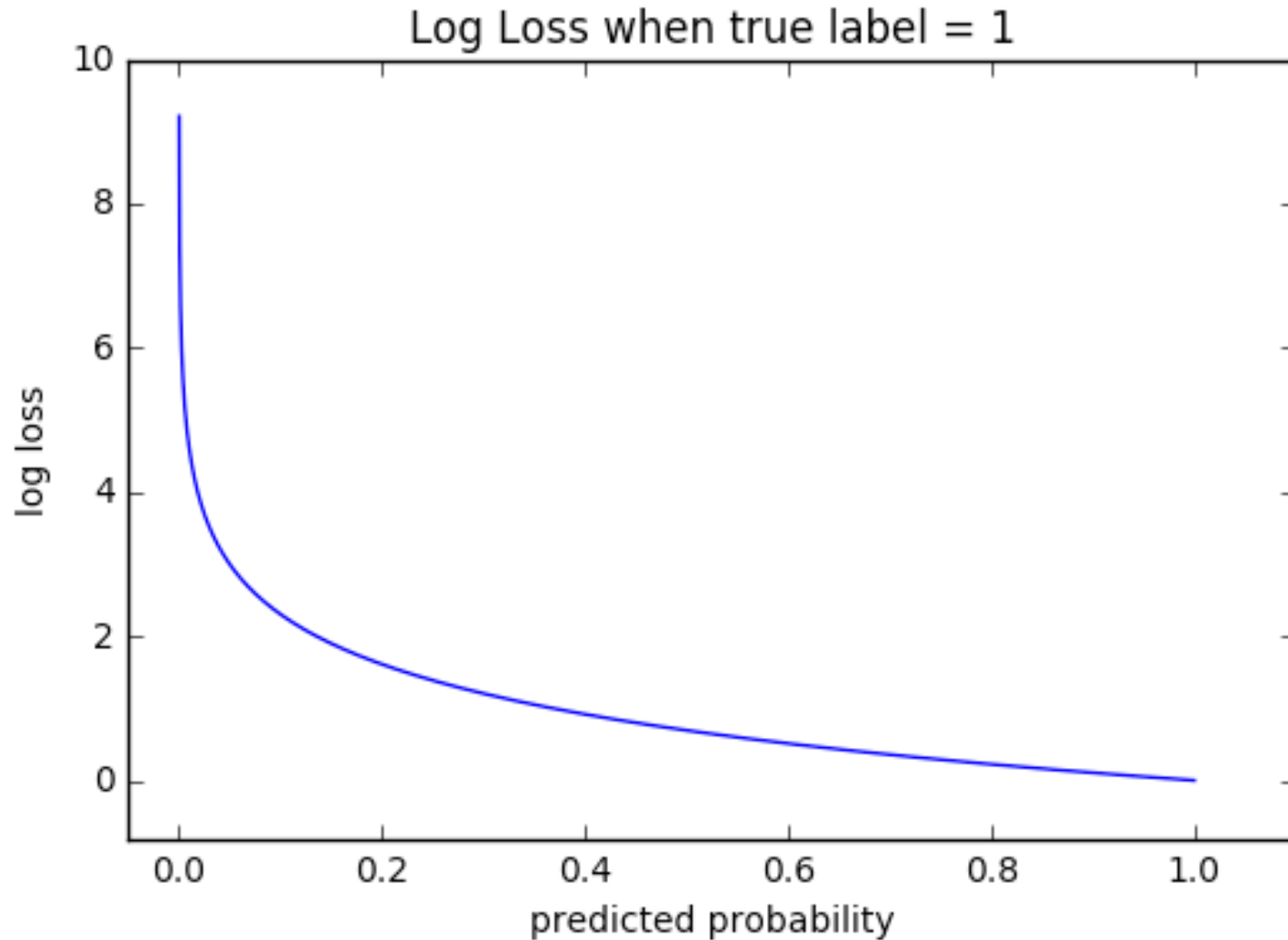
Label

Prob of positive class

Label

Prob of positive class

Why log?



- 1- Cross-Entropy == Negative Log-Likelihood?
- 2- Cross Entropy vs. Sparse Cross Entropy ?

Visualization

<https://playground.tensorflow.org>

Entropy

Self-information

Surprise of an outcome

$$I(x) = \log \frac{1}{P(x)}$$

Entropy

Average surprise

$$H(P) = \sum_i P(x_i) \log \frac{1}{P(x_i)}$$

Cross-Entropy

Average surprise under wrong belief

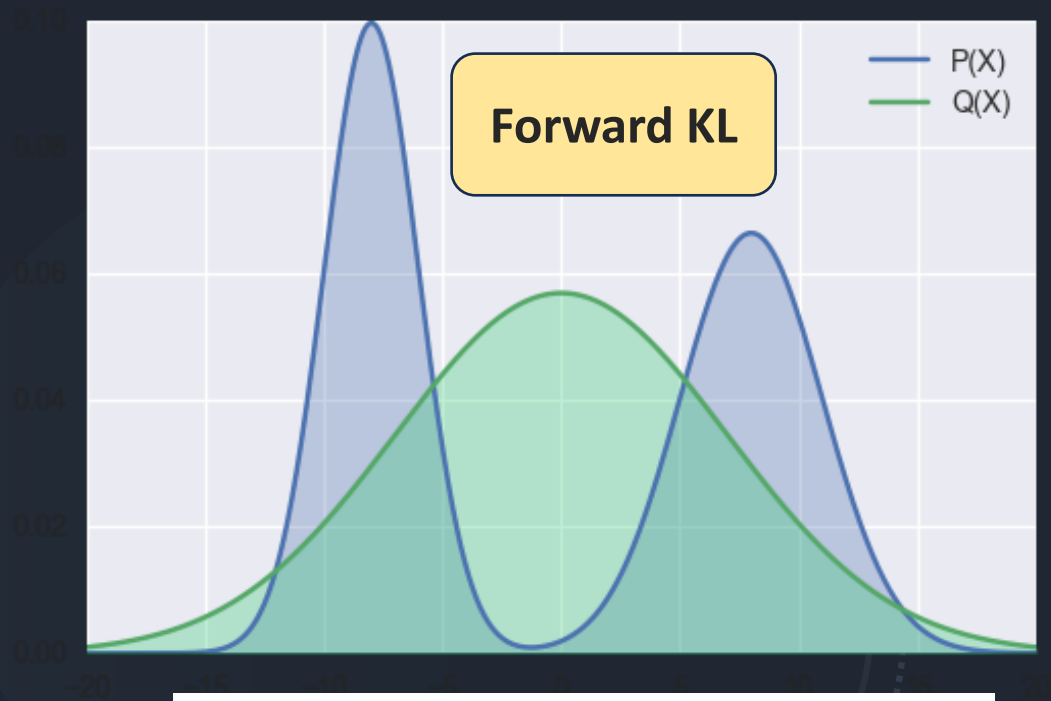
$$H(P, Q) = \sum_i P(x_i) \log \frac{1}{Q(x_i)}$$

KL Divergence

Extra cost for wrong belief

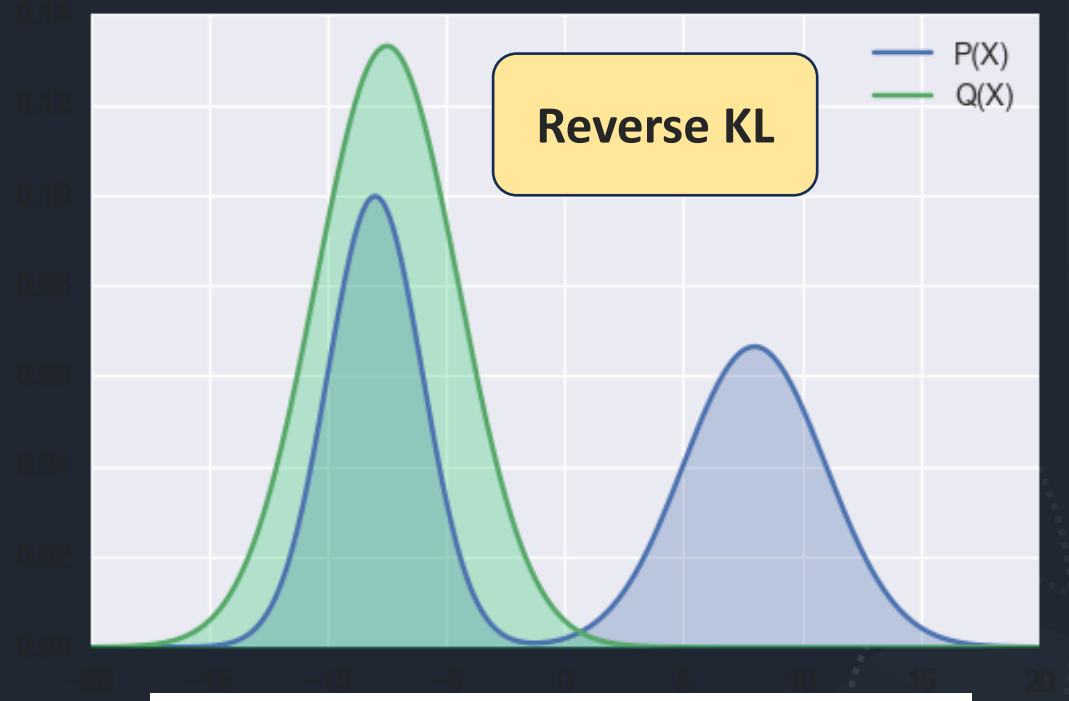
$$D_{\text{KL}}(P \| Q) = \sum_i P(x_i) \log \frac{P(x_i)}{Q(x_i)}$$

Minimizing $KL(p||q)$ or $KL(q||p)$ will give different results



$$KL(p||q) = \sum_{\mathbf{x}} p(\mathbf{x}) \log \frac{p(\mathbf{x})}{q(\mathbf{x})}$$

(zero avoiding)



$$KL(q||p) = \sum_{\mathbf{x}} q(\mathbf{x}) \log \frac{q(\mathbf{x})}{p(\mathbf{x})}$$

(zero forcing)

Minimizing $KL(p||q)$ or $KL(q||p)$ will give different results



Forward KL

Usage:

Classification loss



Reverse KL

Variational inference

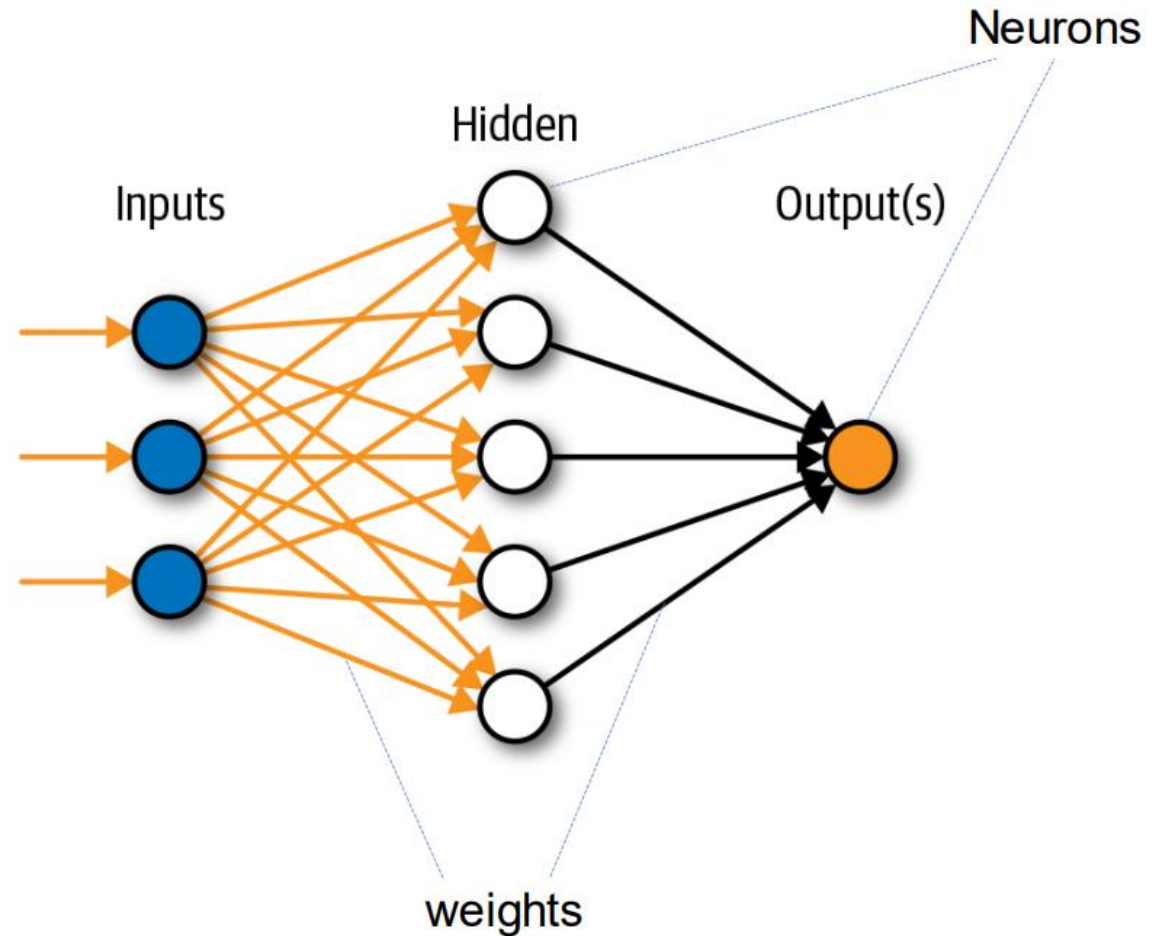
Initialization

- The devil is in the detail



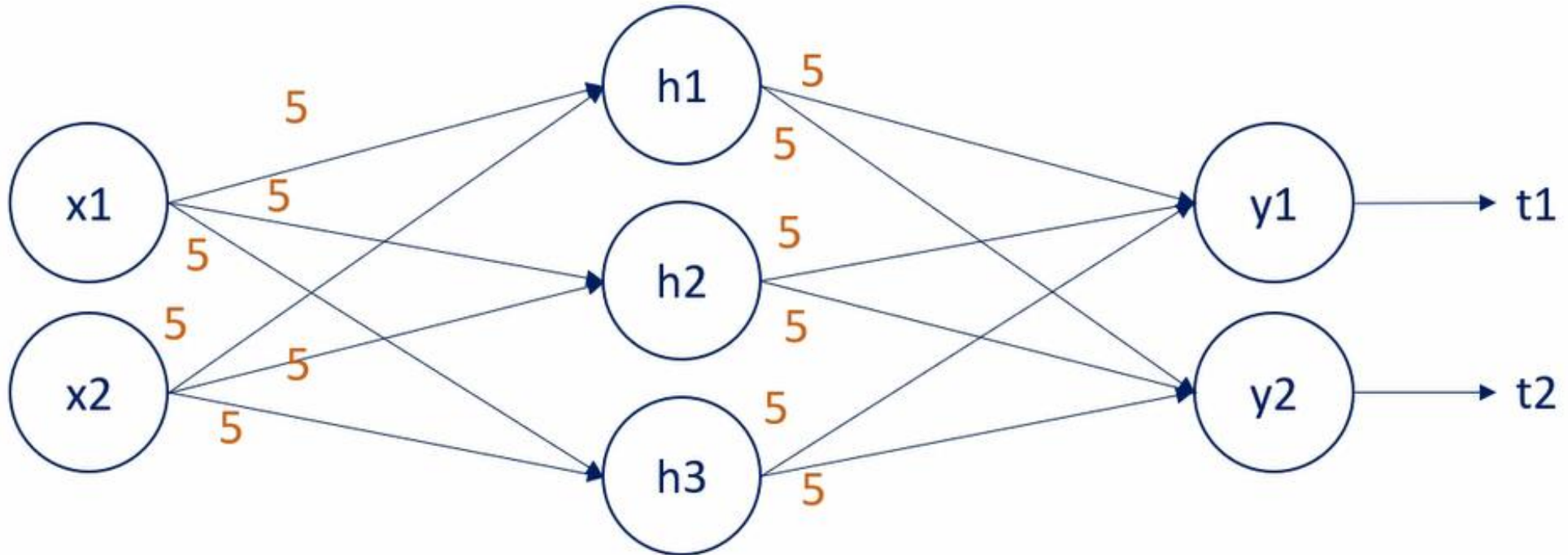
Initialization

- Set the initial values of weights



Constant Initialization

- init with a constant



Random Initialization

- init with a random numbers

```
In [10]: init_range = 0.1

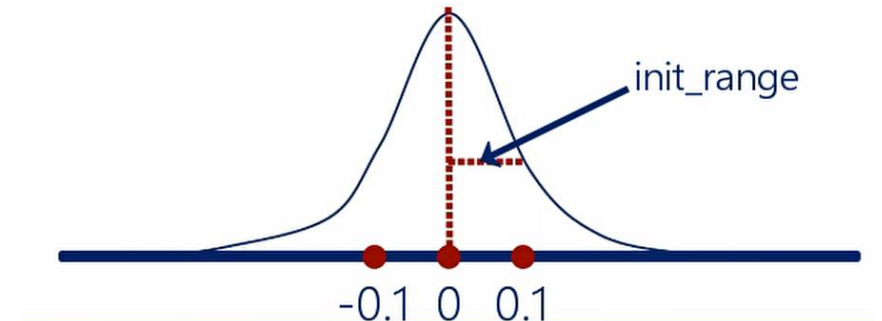
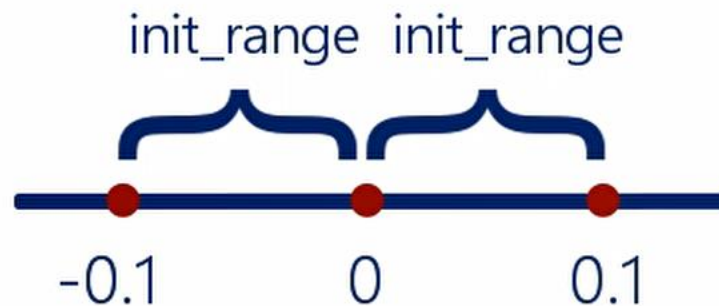
weights = np.random.uniform(-init_range,init_range, size=(2,1))

biases = np.random.uniform(-init_range,init_range,size=1)

print(weights)
print(biases)
```

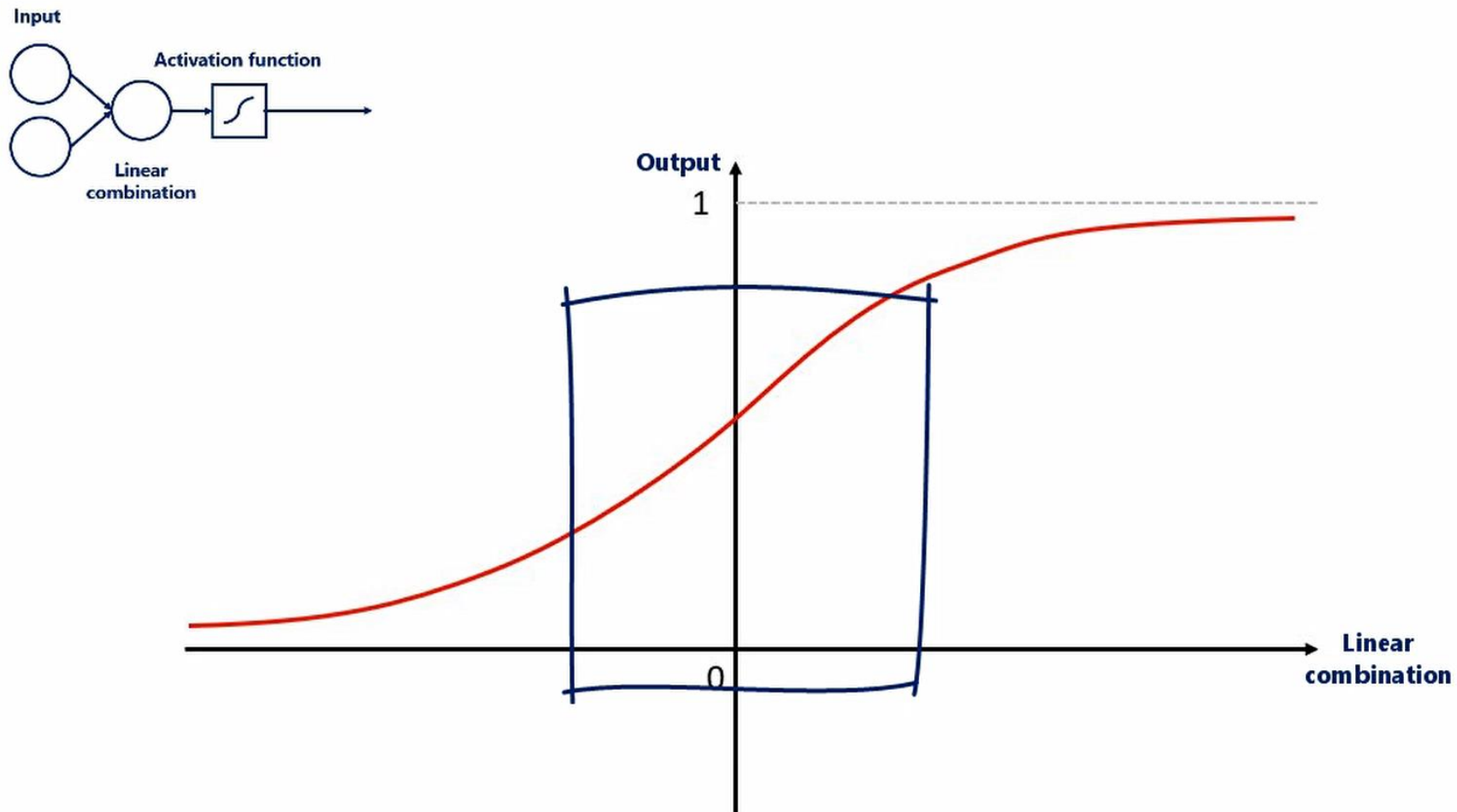
```
[[ -0.02758852]
 [ -0.03556078]]
[-0.05488636]
```

Our initial weights and biases will be picked randomly from the interval $[-0.1, 0.1]$



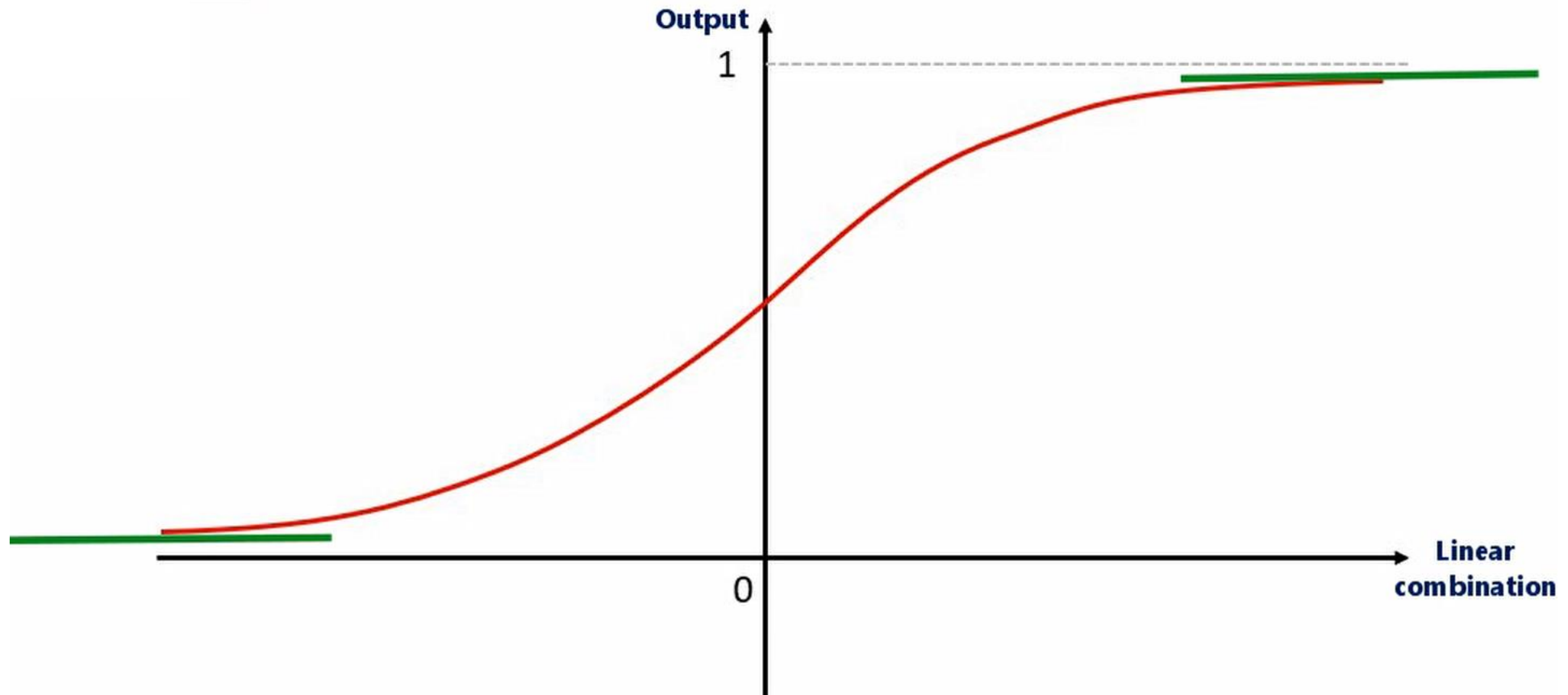
Random Initialization

Problem1 : Non-linearity?



Random Initialization

Problem2 : Flat area



Initialization

Xavier initialization

Xavier Glorot



Glorot initialization

initialization

Original paper

Understanding the difficulty of training deep
feedforward neural networks

*Xavier Glorot, Yoshua Bengio ;
Proceedings of the Thirteenth International Conference on Artificial
Intelligence and Statistics, PMLR 9:249-256, 2010.*

Initialization

- The number of inputs and outputs is a matter! (Method is not so important)

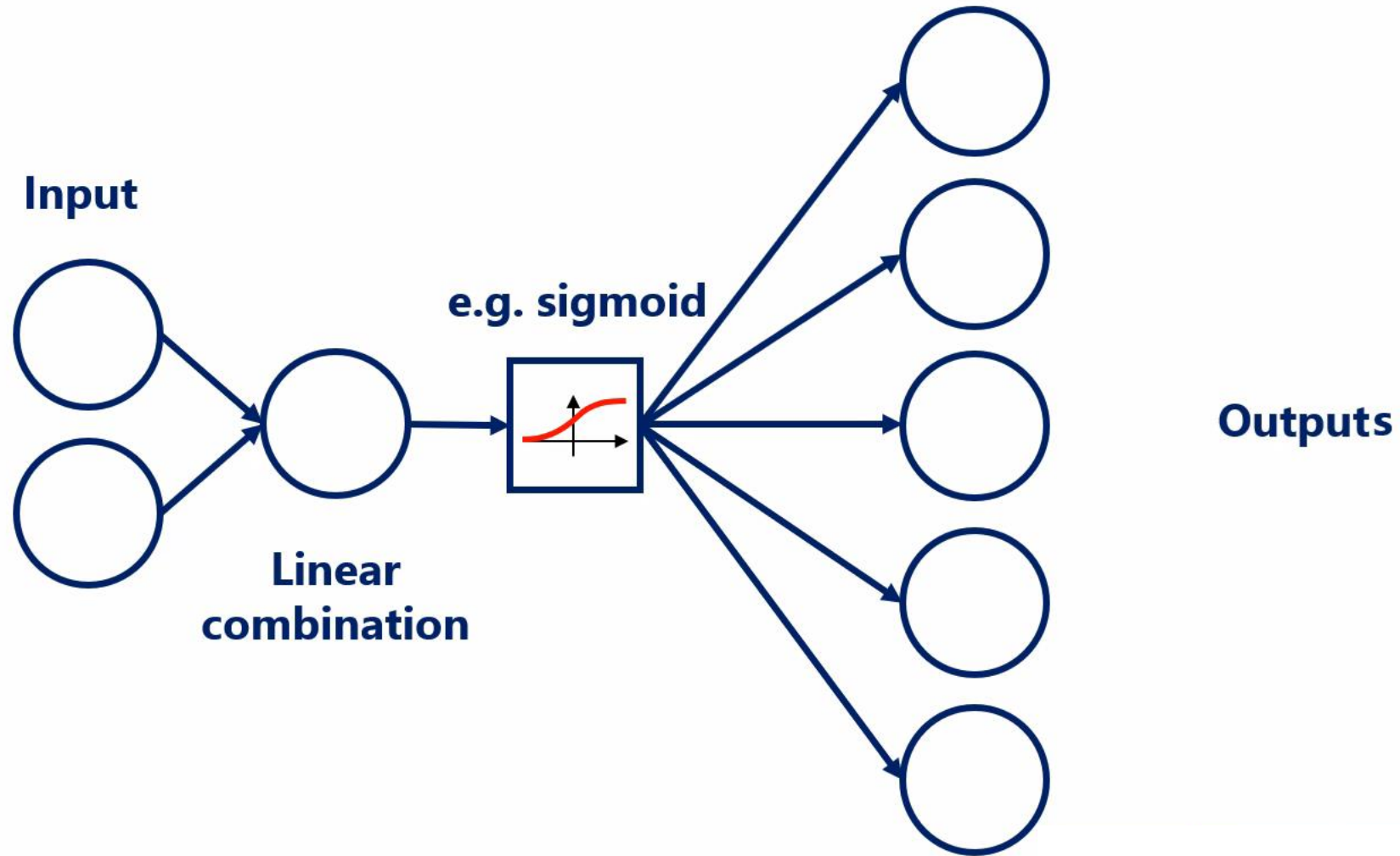
Uniform Xavier initialization: draw each weight, w , from a random uniform distribution

in $[-x, x]$ for $x = \sqrt{\frac{6}{inputs + outputs}}$

Normal Xavier initialization: draw each weight, w , from a normal distribution with a mean

of 0, and a standard deviation $\sigma = \sqrt{\frac{2}{inputs + outputs}}$

Initialization



Initialization

TensorFlow™[Install](#)[Develop](#)[API r1.4](#)[Deploy](#)[Extend](#)[Community](#)[Versions](#)[TFRC](#)

Search

get_collection

get_collection_ref

get_default_graph

get_default_session

get_local_variable

get_seed

```
v1 = foo() # Creates v.  
v2 = foo() # Gets the same, existing v.  
assert v1 == v2
```

If initializer is `None` (the default), the default initializer passed in the variable scope will be used. If that one is `None` too, a `glorot_uniform_initializer` will be used. The initializer can also be a Tensor, in which case the variable is initialized to this value and shape.

Initialization in Conv net

Activation Function	Uniform Distribution [-r, +r]	Normal Distribution
Hyperbolic Tangent Function	$r = \sqrt{\frac{6}{n_{in} + n_{out}}}$	$\sigma = \sqrt{\frac{2}{n_{in} + n_{out}}}$
Sigmoid Function	$r = 4\sqrt{\frac{6}{n_{in} + n_{out}}}$	$\sigma = 4\sqrt{\frac{2}{n_{in} + n_{out}}}$
ReLU (and its variants)	$r = \sqrt{2}\sqrt{\frac{6}{n_{in} + n_{out}}}$	$\sigma = \sqrt{2}\sqrt{\frac{2}{n_{in} + n_{out}}}$

Initialization in Conv net

tf.keras.initializers.HeUniform 



View source on GitHub

He uniform variance scaling initializer.

Inherits From: `VarianceScaling`, `Initializer`



View aliases

```
tf.keras.initializers.HeUniform(  
    seed=None  
)
```



Also available via the shortcut function `tf.keras.initializers.he_uniform`.

Draws samples from a uniform distribution within `[-limit, limit]`, where `limit = sqrt(6 / fan_in)` (`fan_in` is the number of input units in the weight tensor).

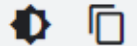
Initialization in Conv net

Examples:

```
>>> # Standalone usage:  
>>> initializer = tf.keras.initializers.HeUniform()  
>>> values = initializer(shape=(2, 2))
```



```
>>> # Usage in a Keras layer:  
>>> initializer = tf.keras.initializers.HeUniform()  
>>> layer = tf.keras.layers.Dense(3, kernel_initializer=initializer)
```



Other Initialization Methods

▼ initializers

Overview

Constant

GlorotNormal

GlorotUniform

HeNormal

HeUniform

Identity

Initializer

LecunNormal

LecunUniform

Ones

Orthogonal

RandomNormal

RandomUniform

TruncatedNormal

VarianceScaling

Zeros

deserialize

get

serialize

https://www.tensorflow.org/api_docs/python/tf/keras/initializers

<https://keras.io/api/layers/initializers/>

Learning Rate

Hyperparameters

pre-set by us

vs

Parameters

found by optimizing

Width

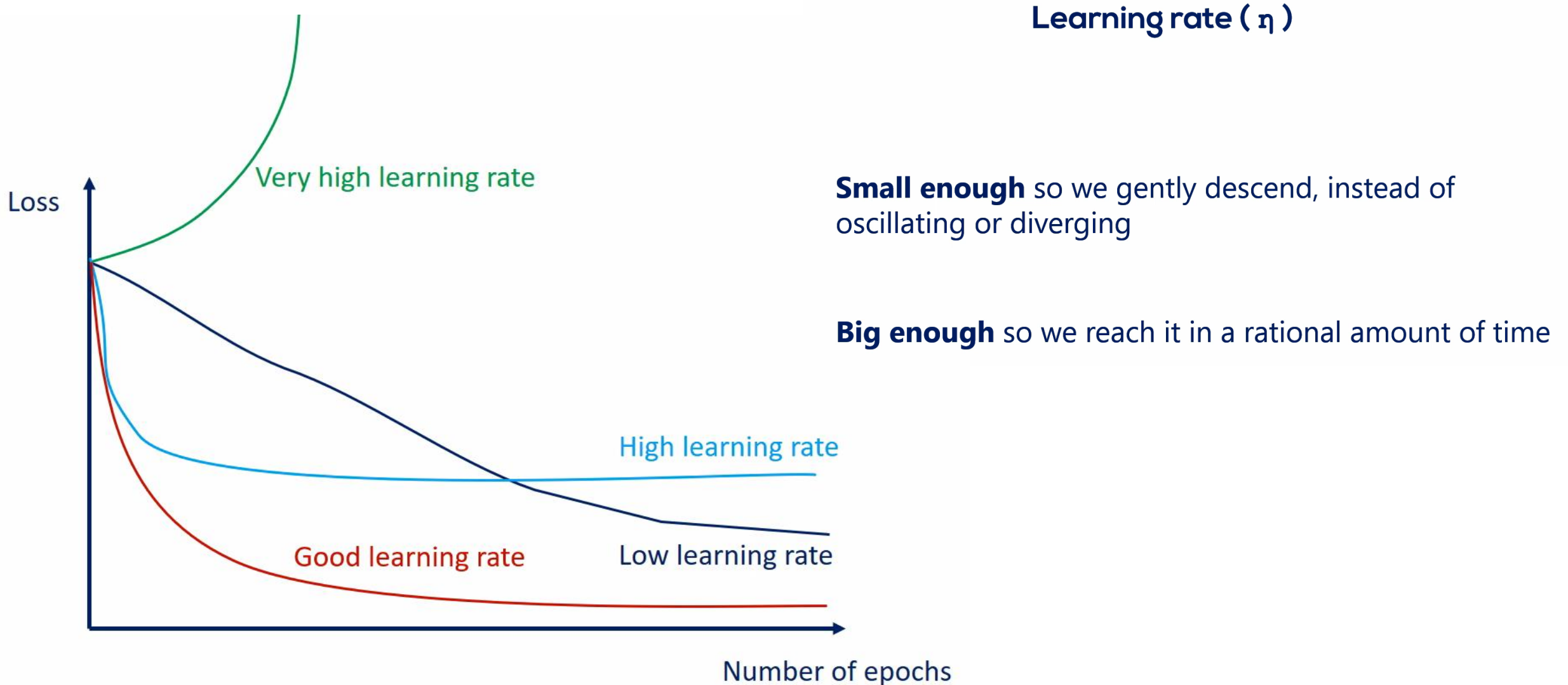
Depth

Learning rate (η)

Weights (w)

Biases (b)

Setting Learning Rate



Learning Rate Schedules

1. We start from a high initial learning rate

2. At some point we lower the rate to avoid oscillation

3. Around the end we pick a very small rate to get a precise answer

$$\eta_0 = 0.1$$

$$\eta_1 = 0.0967$$

$$\eta_2 = 0.0905$$

$$\eta_3 = 0.0819$$

$$\eta_4 = 0.0717$$

$$\eta_5 = 0.0607$$

$$\eta_6 = 0.0497$$

$$\eta_7 = 0.0393$$

$$\eta_8 = 0.0301$$

current epoch

$$\eta = \eta_0 e^{-n/20}$$

some constant

Learning Rate Schedules

$$\eta_0 = 0.1$$

$$\eta_1 = 0.0967$$

$$\eta_2 = 0.0905$$

$$\eta_3 = 0.0819$$

$$\eta_4 = 0.0717$$

$$\eta_5 = 0.0607$$

$$\eta_6 = 0.0497$$

$$\eta_7 = 0.0393$$

$$\eta_8 = 0.0301$$

current epoch

$$\eta = \eta_0 e^{-n/20}$$

some constant

c is yet another hyperparameter

No set rule, but same order of magnitude

e.g. if we need:

100 epochs, $50 < c < 500$

1000 epochs, $500 < c < 5000$

$c \sim 20$ is ok

Learning Rate Schedules in TF

 TensorFlow

Install

Learn ▾

API ▾

Resources ▾

Community ▾

Why TensorFlow ▾

Search

Overview

Python

C++

Java

More

Filter

experimental

legacy

schedules

Overview

CosineDecay

CosineDecayRestarts

ExponentialDecay

InverseTimeDecay

LearningRateSchedule

PiecewiseConstantDecay

PolynomialDecay

deserialize

serialize

preprocessing

regularizers

utils

tf.linalg

Used in the tutorials

- Neural machine translation with a Transformer and Keras

You can use a learning rate schedule to modulate how the learning rate of your optimizer changes over time.

Several built-in learning rate schedules are available, such as `tf.keras.optimizers.schedules.ExponentialDecay` or `tf.keras.optimizers.schedules.PiecewiseConstantDecay`:

```
lr_schedule = keras.optimizers.schedules.ExponentialDecay(
    initial_learning_rate=1e-2,
    decay_steps=10000,
    decay_rate=0.9)
optimizer = keras.optimizers.SGD(learning_rate=lr_schedule)
```

A `LearningRateSchedule` instance can be passed in as the `learning_rate` argument of any optimizer.

To implement your own schedule object, you should implement the `__call__` method, which takes a `step` argument (scalar integer tensor, the current training step count). Like for any other Keras object, you can also optionally make your object serializable by implementing the `get_config` and `from_config` methods.

Learning Rate Schedules

```
# Custom Scheduler Function
lr_start  = 1e-4
lr_max    = 0.000015 * replicas * batch_size
lr_min    = 1e-7
r_ramp_ep = 3
lr_sus_ep  = 0
lr_decay  = 0.7

def lr_fn(epoch):
    if epoch < lr_ramp_ep:
        lr = (lr_max - lr_start) / lr_ramp_ep * epoch + lr_start

    elif epoch < lr_ramp_ep + lr_sus_ep:
        lr = lr_max

    else:
        lr = (lr_max - lr_min) * lr_decay**(epoch - lr_ramp_ep - lr_sus_ep) + lr_min

    return lr

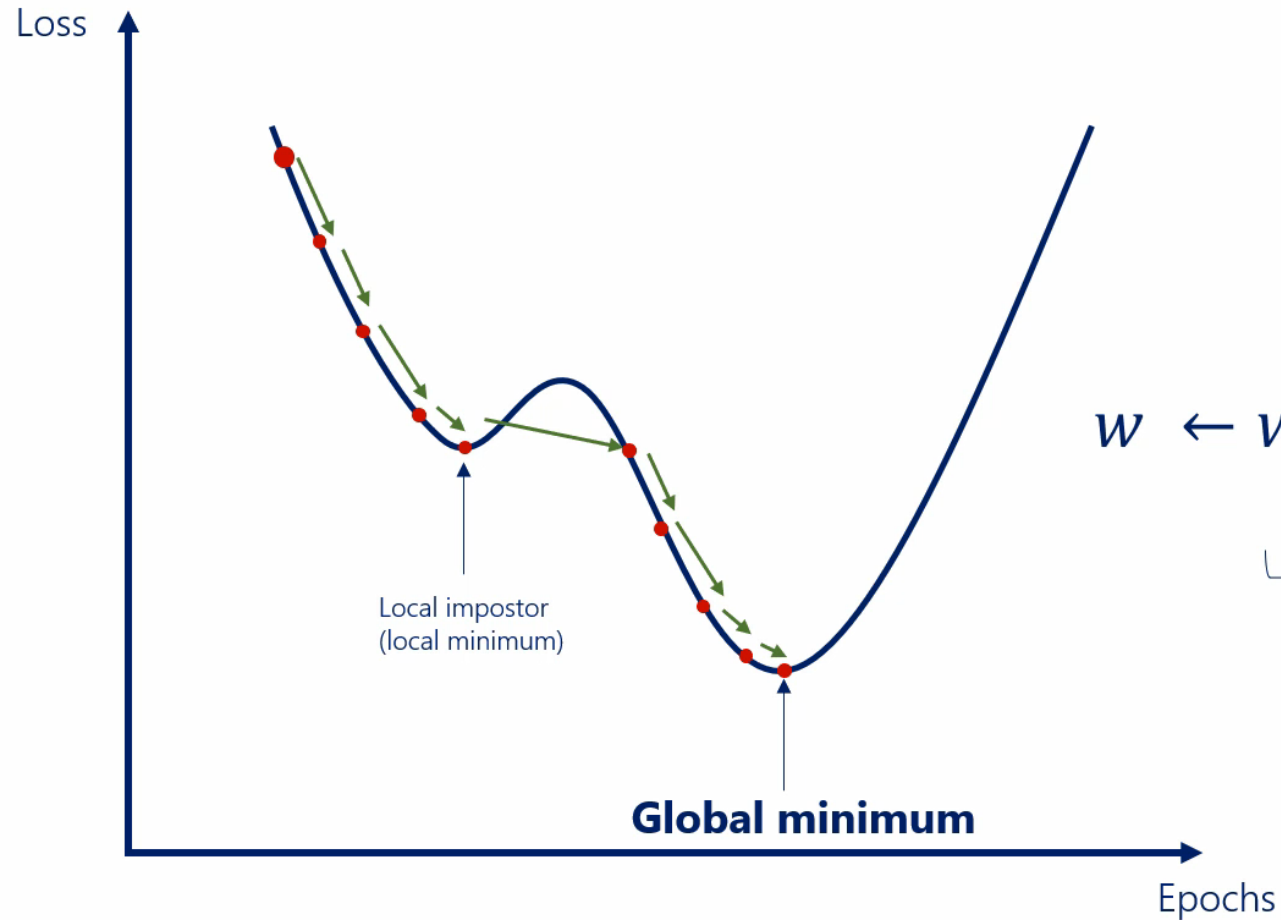
# Using this Custom Function, create a Callback
lr_callback = tf.keras.callbacks.LearningRateScheduler(lr_fn, verbose=True)

# Train the Model
model.fit(..., callbacks=[lr_callback, WandbCallback()])
```

```
>>> # This function keeps the initial learning rate for the first ten epochs
>>> # and decreases it exponentially after that.
>>> def scheduler(epoch, lr):
...     if epoch < 10:
...         return lr
...     else:
...         return lr * tf.math.exp(-0.1)
>>>
>>> model = tf.keras.models.Sequential([tf.keras.layers.Dense(10)])
>>> model.compile(tf.keras.optimizers.SGD(), loss='mse')
>>> round(model.optimizer.lr.numpy(), 5)
0.01
```

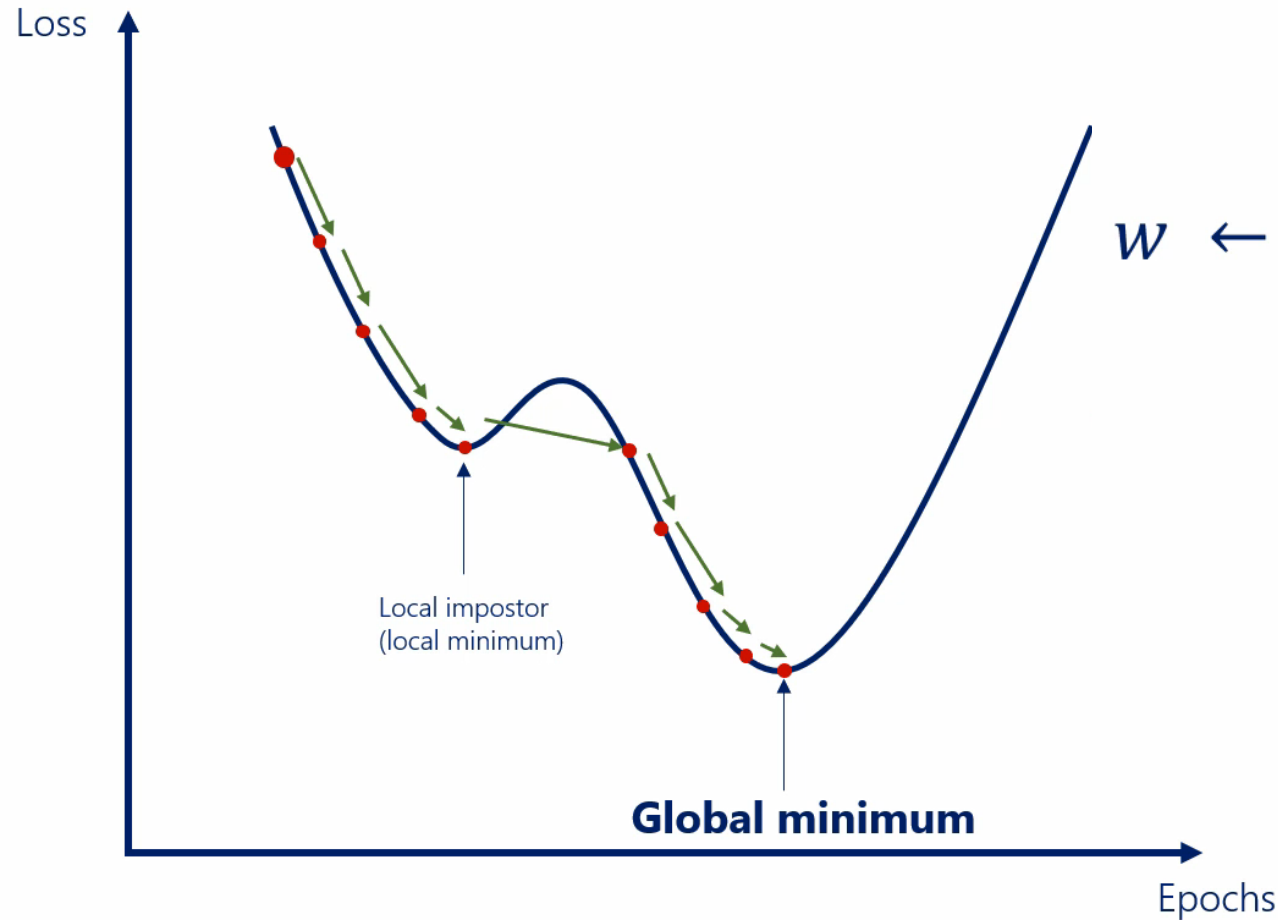
```
>>> callback = tf.keras.callbacks.LearningRateScheduler(scheduler)
>>> history = model.fit(np.arange(100).reshape(5, 20), np.zeros(5),
...                     epochs=15, callbacks=[callback], verbose=0)
>>> round(model.optimizer.lr.numpy(), 5)
0.00607
```

Momentum



$$w \leftarrow \underbrace{w(t) - \eta \frac{\partial L}{\partial w}(t)}_{\text{Current update}} - \underbrace{\eta \frac{\partial L}{\partial w}(t-1)}_{\text{Update a moment ago}}$$

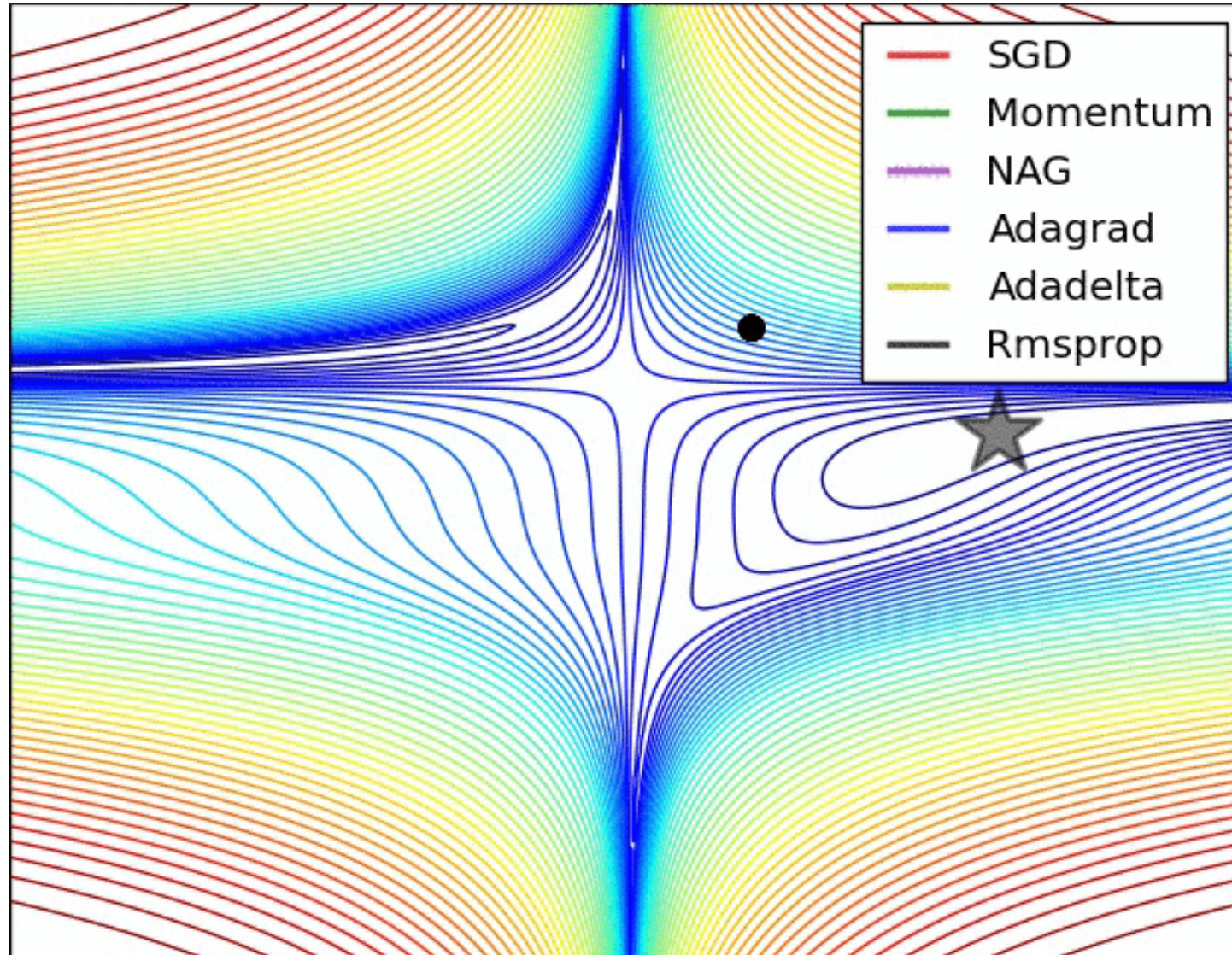
Momentum



$$w \leftarrow w(t) - \underbrace{\eta \frac{\partial L}{\partial w}(t)}_{\text{Current update}} - \underbrace{\alpha \eta \frac{\partial L}{\partial w}(t-1)}_{\text{Update a moment ago}}$$

$\alpha = 0.9$ is conventional

Momentum



Categorical data

How to encode categories in a way useful for ML



One-hot encoding



Binary encoding

Binary Encoding

	ordinal	binary
	1	01
	2	10
	3	11

Binary Encoding

	ordinal	var 1	var 2
	1	0	1
	2	1	0
	3	1	1

By splitting the numbers in two variable we have removed the order ...

but ...

Binary Encoding

	ordinal	var 1	var 2
	1	0	1
	2	1	0
	3	1	1

- Bread is the opposite of yogurt ?

One-hot Encoding

	bread	yogurt	muffins
	✓		
		✓	
			✓



	bread	yogurt	muffins
	1	0	0
	0	1	0
	0	0	1

One-hot Encoding

Problem?



12,000,000 products



12,000,000 Columns ?

Categorical data

	ONE-HOT	BINARY
NEW COLUMNS	12,000	16

12,000 IN BINARY: 10111011100000

Better!

Categorical data

One-hot



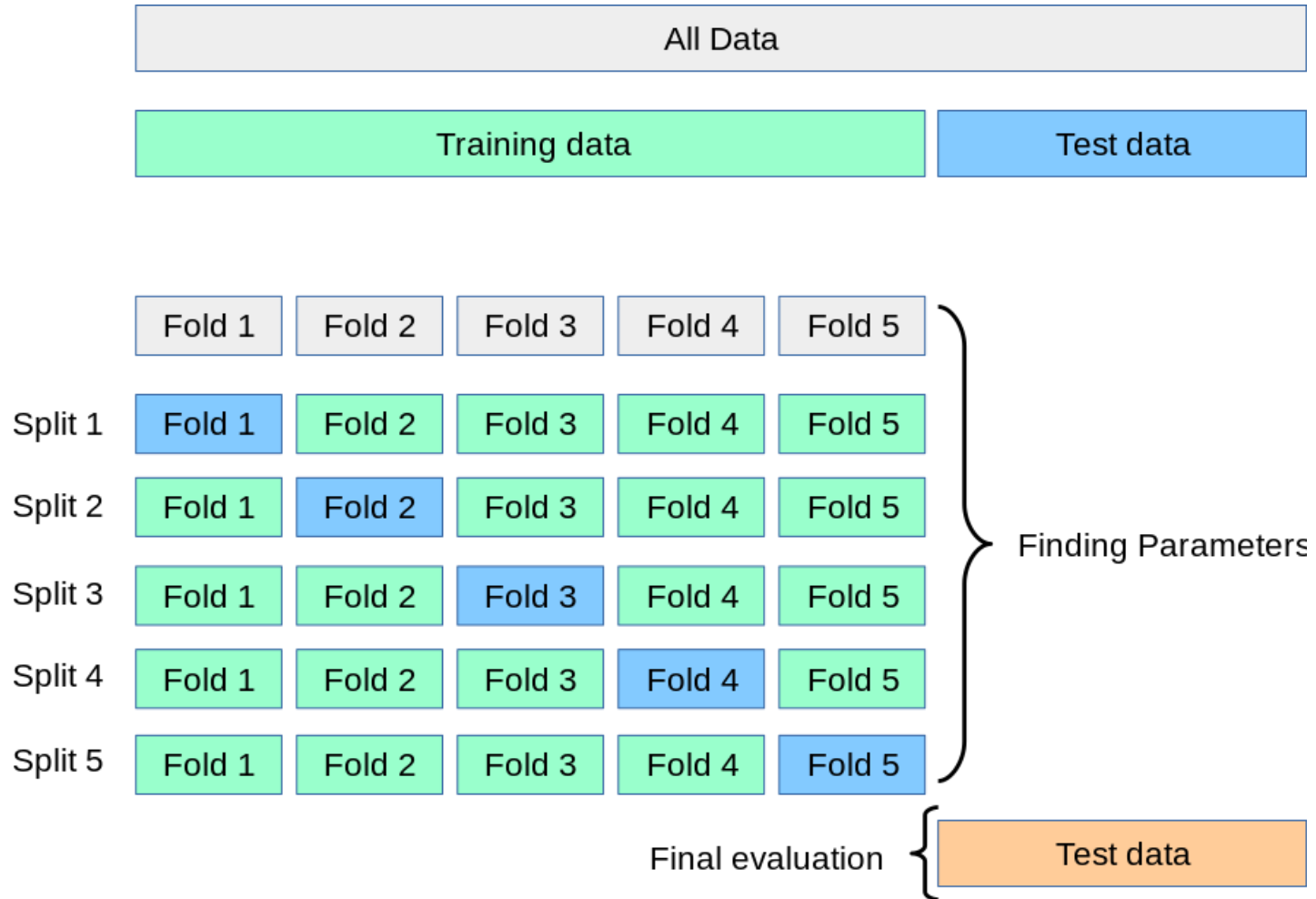
Few categories

Binary



Many categories

Cross Validation



Cross-entropy loss

The diagram illustrates the derivation of the binary cross-entropy loss formula. It starts with the general cross-entropy formula for M classes:

$$-\sum_{j=1}^M y_j \log(p(y_j))$$

Annotations for this formula:

- Indicator variable**: Points to y_j .
- Prob of class j** : Points to $p(y_j)$.

A blue bracket under the summation indicates the **Sum over classes**. A pink bracket under the summation index i in the formula below indicates the **Sum over trials**.

The formula then expands to the binary case for N trials:

$$-\sum_{i=1}^N y_i \log(p(y_i)) + (1 - y_i) \log(1 - p(y_i))$$

Annotations for this formula:

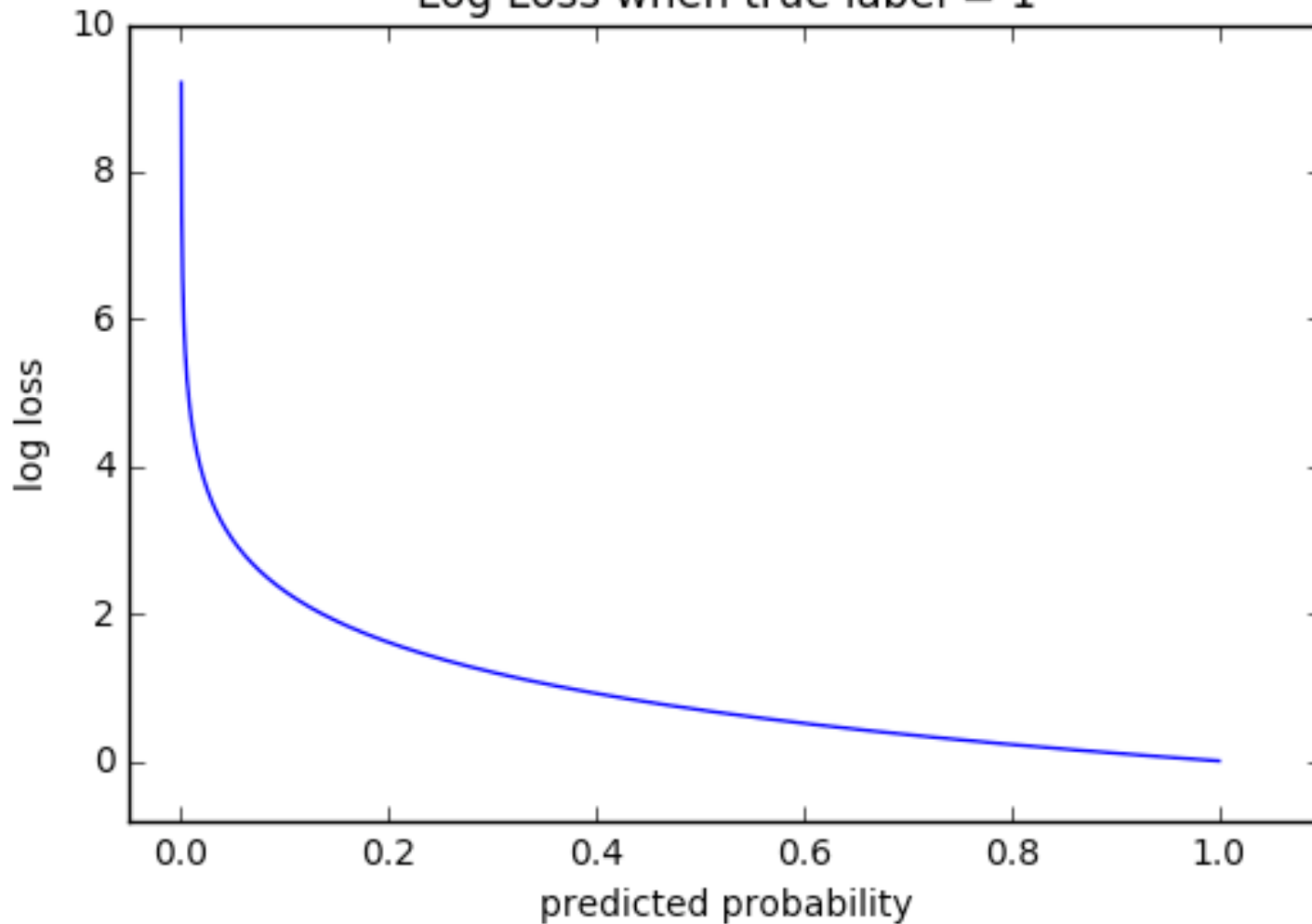
- Label**: Points to y_i in the first term.
- Prob of positive class**: Points to $p(y_i)$ in the first term.
- Label**: Points to y_i in the second term.
- Prob of positive class**: Points to $1 - p(y_i)$ in the second term.

A blue bracket under the entire expression indicates the **Sum over trials**.

Your Turn



Log Loss when true label = 1



1- Cross-Entropy == Negative Log-Likelihood?

2- Cross Entropy vs. Sparse Cross Entropy ?

Coding Session

- Training a simple MLP network

